

"Authenticate the identity, authorize the action, minimize the permissions."

MODULE 12

Security & Compliance

Protecting identities, APIs, secrets and microservices.

Zero Trust principles

OAuth 2.0 and OpenID Connect

Keycloak / Microsoft Entra ID

JWT access tokens

Service-to-service security

Secrets management

API security

DevSecOps, audit and compliance

LEARNING OBJECTIVES

Identity, OAuth2/OIDC, secrets, API protection and compliance.



BY THE END OF THIS MODULE, YOU CAN

1. Explain microservices security challenges
2. Apply Zero Trust principles
3. Differentiate authentication and authorization
4. Explain OAuth 2.0 and OpenID Connect
5. Understand common OAuth flows
6. Interpret JWT claims
7. Protect Spring Boot APIs
8. Integrate Keycloak or Microsoft Entra ID
9. Secure service-to-service calls
10. Manage secrets safely
11. Apply API security controls
12. Integrate security into CI/CD
13. Support auditing and compliance



FIVE SECTIONS

1

Security foundations

Identity · Trust · Authentication · Authorization

2

OAuth2 / OIDC

Identity provider → Token → Protected API

3

Distributed security

Users · Services · Agents · MCP servers · CI/CD workloads

4

Secrets and APIs

Credentials · Certificates · API keys · Database passwords

5

Continuous security

Code → Build → Container → Deployment → Runtime → Audit

KEY TAKEAWAY Security is a cross-cutting concern across every service, identity, API and deployment.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Authentication

Proving who a caller is.

2 Authorization

Deciding what an authenticated caller is allowed to do.

3 Zero Trust

Never trust a request because of its network location — verify every one.

4 OAuth 2.0

A standard for granting limited, token-based access without sharing passwords.

5 OpenID Connect

An identity layer on OAuth 2.0 that tells the application who signed in.

6 JSON Web Token

A signed token carrying claims such as subject, issuer, audience and scopes.

7 Scope

A permission in a token, such as accounts.read, that limits what the caller may do.

8 Secret

A sensitive value, such as a password or key, kept out of code and injected at runtime.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

AI	Artificial Intelligence	HTTPS	HyperText Transfer Protocol Secure	OIDC	OpenID Connect
API	Application Programming Interface	ID	Identifier	PKCE	Proof Key for Code Exchange
CI	Continuous Integration	IdP	Identity Provider	RBAC	Role-Based Access Control
CI/CD	Continuous Integration / Continuous Delivery	JWKS	JSON Web Key Set	SAST	Static Application Security Testing
CORS	Cross-Origin Resource Sharing	JWT	JSON Web Token	SBOM	Software Bill of Materials
DB	Database	MCP	Model Context Protocol	SQL	Structured Query Language
DTO	Data Transfer Object	MDC	Mapped Diagnostic Context	TLS	Transport Layer Security
HMAC	Hash-based Message Authentication Code	mTLS	Mutual Transport Layer Security	URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol	OAuth	Open Authorization	WAF	Web Application Firewall

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 11 TO MODULE 12

Users, services, agents and pipelines all call our APIs — secure every one of them.

MODULES 1–11 GAVE US

- ✓ Many services and APIs
- ✓ Events, gateways and OpenShift
- ✓ Pipelines holding deploy rights
- ✓ AI agents calling tools
- ✓ MCP servers with per-tool scopes

NOW Give every identity, token, secret and API explicit, auditable trust.



OUR PATH THROUGH MODULE 12

- 1 Why microservices security differs
- 2 OAuth2, OIDC and PKCE
- 3 JWT validation, scopes and roles
- 4 Gateways and service identities
- 5 Secrets and API protection
- 6 Audit, logging and compliance
- 7 DevSecOps and the secure architecture

Part 1

Part 2

Part 2

Part 3

Part 4

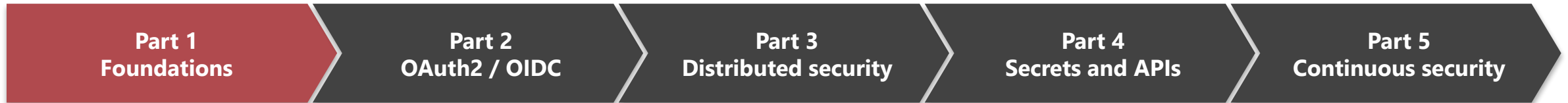
Part 5

Part 5

KEY TAKEAWAY Every earlier module created a new identity or boundary — Module 12 secures them all.

PART 1: SECURITY FOUNDATIONS

Why distributed systems need a different security mindset.



SO FAR

- ✓ Every service exposes an API
- ✓ Agents and pipelines call those APIs
- ✓ Tokens and scopes appeared along the way

NEXT QUESTION

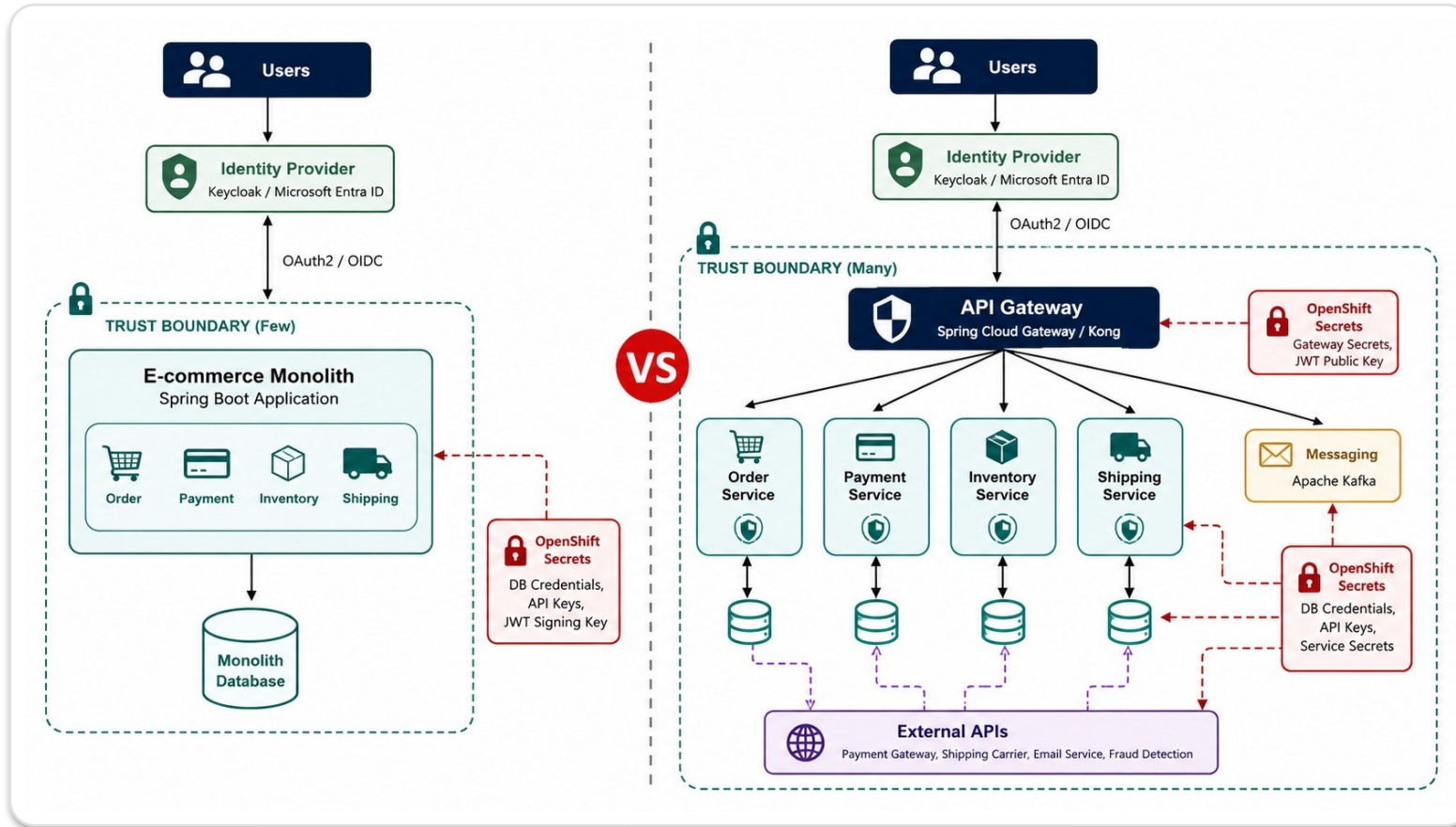
Why can't we secure microservices the way we secured a monolith?

IN THIS PART

- 1 Why microservices security is different
- 2 Zero Trust
- 3 Authentication vs. authorization
- 4 Exercise: 401 or 403?

WHY MICROSERVICES SECURITY IS DIFFERENT

A larger attack surface, distributed trust, many APIs and identities.



QUESTIONS AT EVERY HOP

- ? Who is calling?
- ? With which identity?
- ? What permission allows it?
- ? Can we prove it later?

More credentials

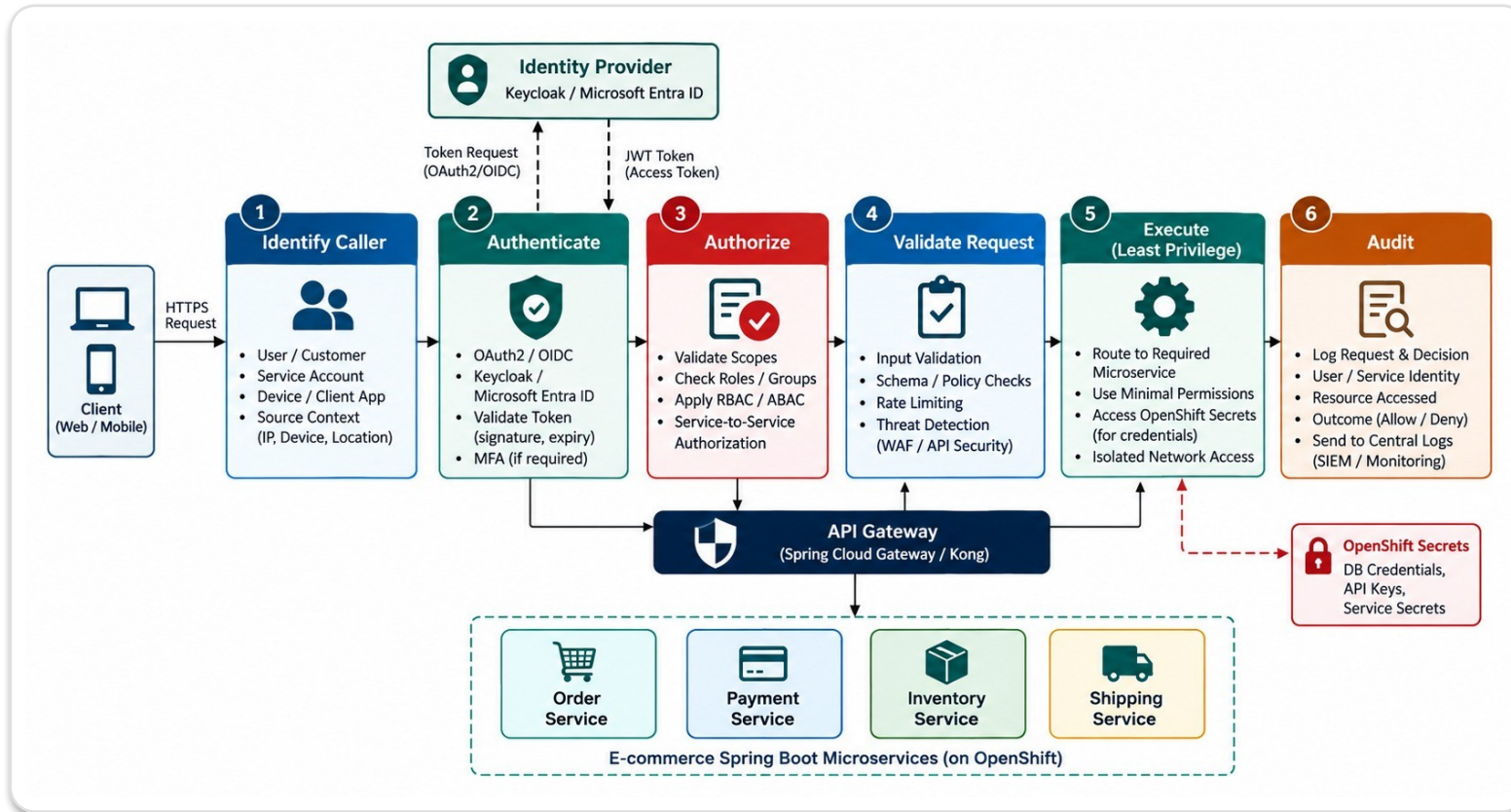
Database passwords, service secrets, tokens and keys — per service.

TAKEAWAY Protect every interaction, not just the edge.

KEY TAKEAWAY Microservices security is about identity, controlled trust and protecting every interaction.

ZERO TRUST SECURITY FOR MICROSERVICES

Never trust a request based only on its network location.



Old thinking

"It's inside our network, so it's safe."

Zero Trust

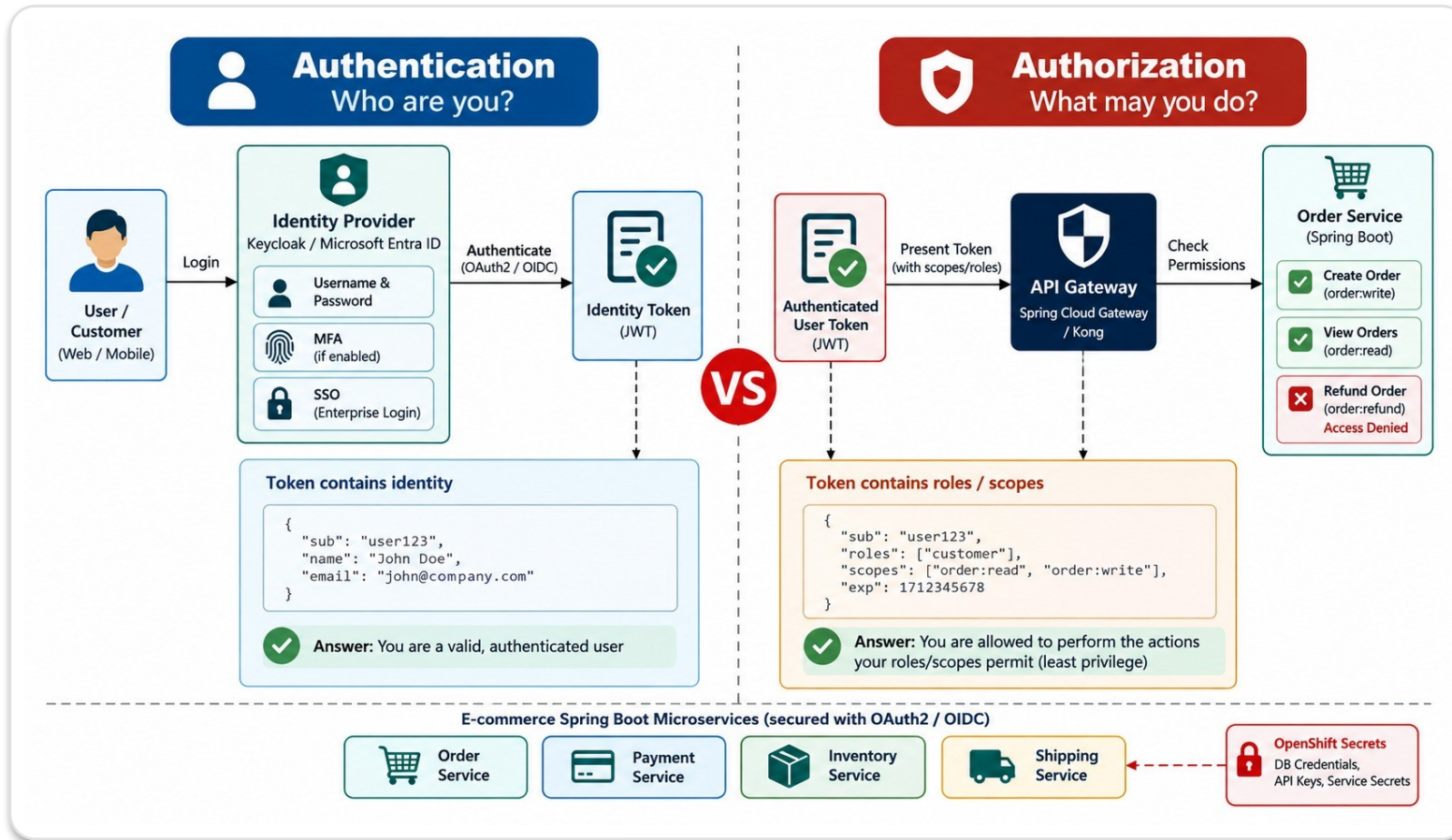
Verify identity, permission and policy on every request.

REMEMBER Never trust, always verify.

KEY TAKEAWAY Trust comes from identity, permission and policy — never from network location alone.

AUTHENTICATION VS. AUTHORIZATION

Who are you? versus what may you do?



Authentication

Proves identity — a valid, unexpired token from a trusted issuer.

Authorization

Decides access — scope, role and ownership of this object.

HTTP tells them apart

401 Unauthorized → not authenticated
403 Forbidden → not allowed

KEY TAKEAWAY Being authenticated never means being allowed to do everything.

EXERCISE: 401 OR 403?

Decide the right response for each request to Account Service.

PRACTICE EXERCISE

8 MIN

SCENARIO

GET /api/v1/accounts/ACC-5001 is owned by customer CUST-0007.

- A no Authorization header
- B expired token
- C valid token, scope transactions.read
- D valid token, accounts.read, sub CUST-0009
- E valid token, accounts.read, sub CUST-0007
- F token signed by an unknown issuer

YOUR TASKS

- 1 Give the status code for each request
- 2 Say whether it's authentication or authorization
- 3 Decide what gets logged — and what never does

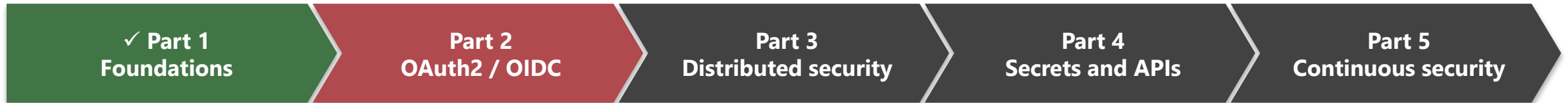
EXPECTED OUTCOME

- ✓ A, B, F → 401 (authentication)
- ✓ C → 403 (wrong scope)
- ✓ D → 403 or 404 (not the owner)
- ✓ E → 200
- ✓ Log subject and decision — never the token

KEY TAKEAWAY 401 means we don't know who you are; 403 means we know — and the answer is no.

PART 2: OAUTH2 AND OPENID CONNECT

Tokens instead of passwords — and how Spring Boot validates them.



SO FAR

- ✓ Every hop can be an attack path
- ✓ Zero Trust verifies every request
- ✓ 401 is authentication; 403 is authorization

NEXT QUESTION

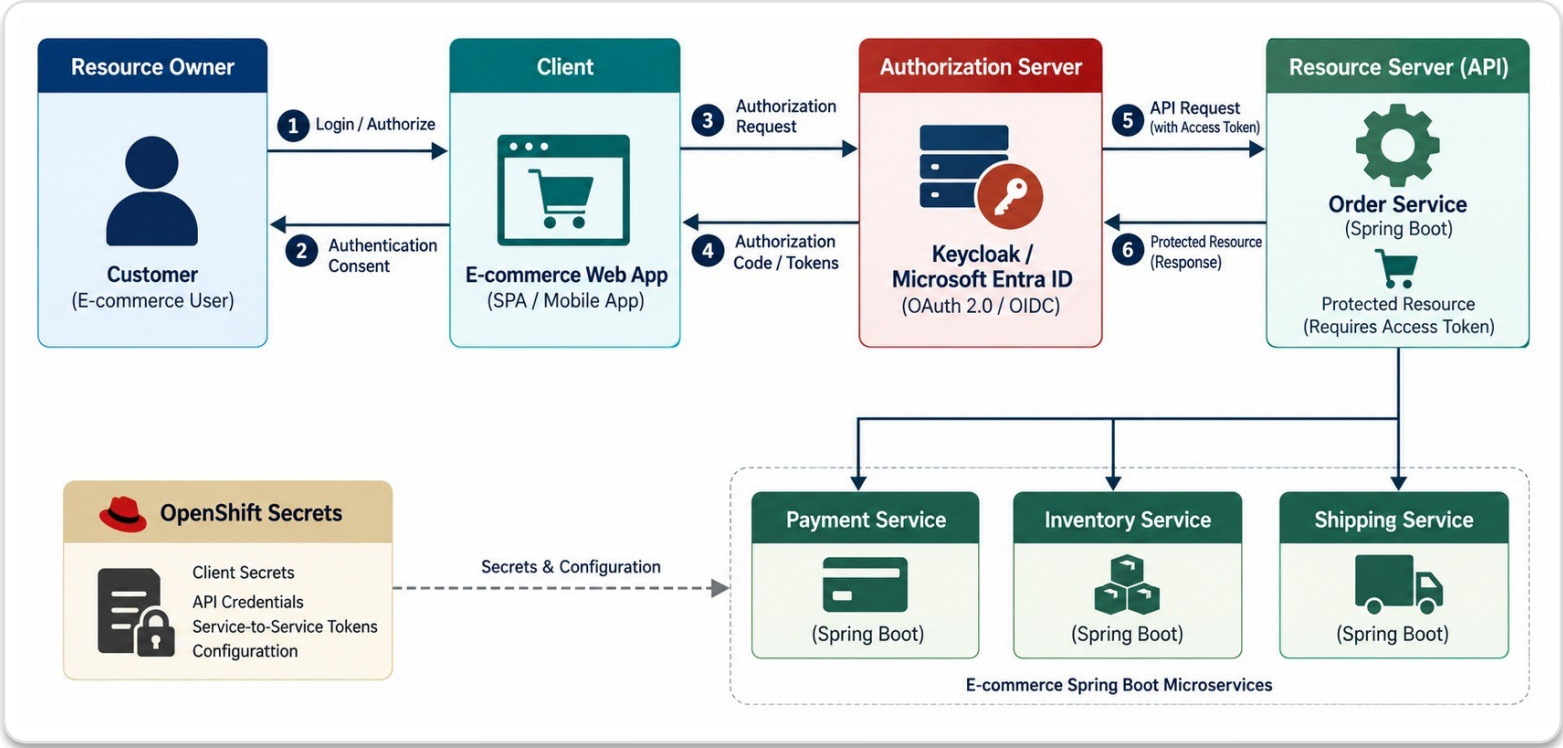
How do users and clients get tokens — and how does a service trust them?

IN THIS PART

- 1 OAuth 2.0, OIDC and how they differ
- 2 The architecture and Authorization Code + PKCE
- 3 JWTs — and validating them in code
- 4 Roles, scopes, claims — and object-level checks
- 5 Exercise 3.2: JWT roles and scopes
- 6 Keycloak and Microsoft Entra ID

OAUTH 2.0 FUNDAMENTALS

Roles, access tokens, scopes and authorization flows.



Resource owner
 The user who owns the data.

Client
 The app acting on the user's behalf.

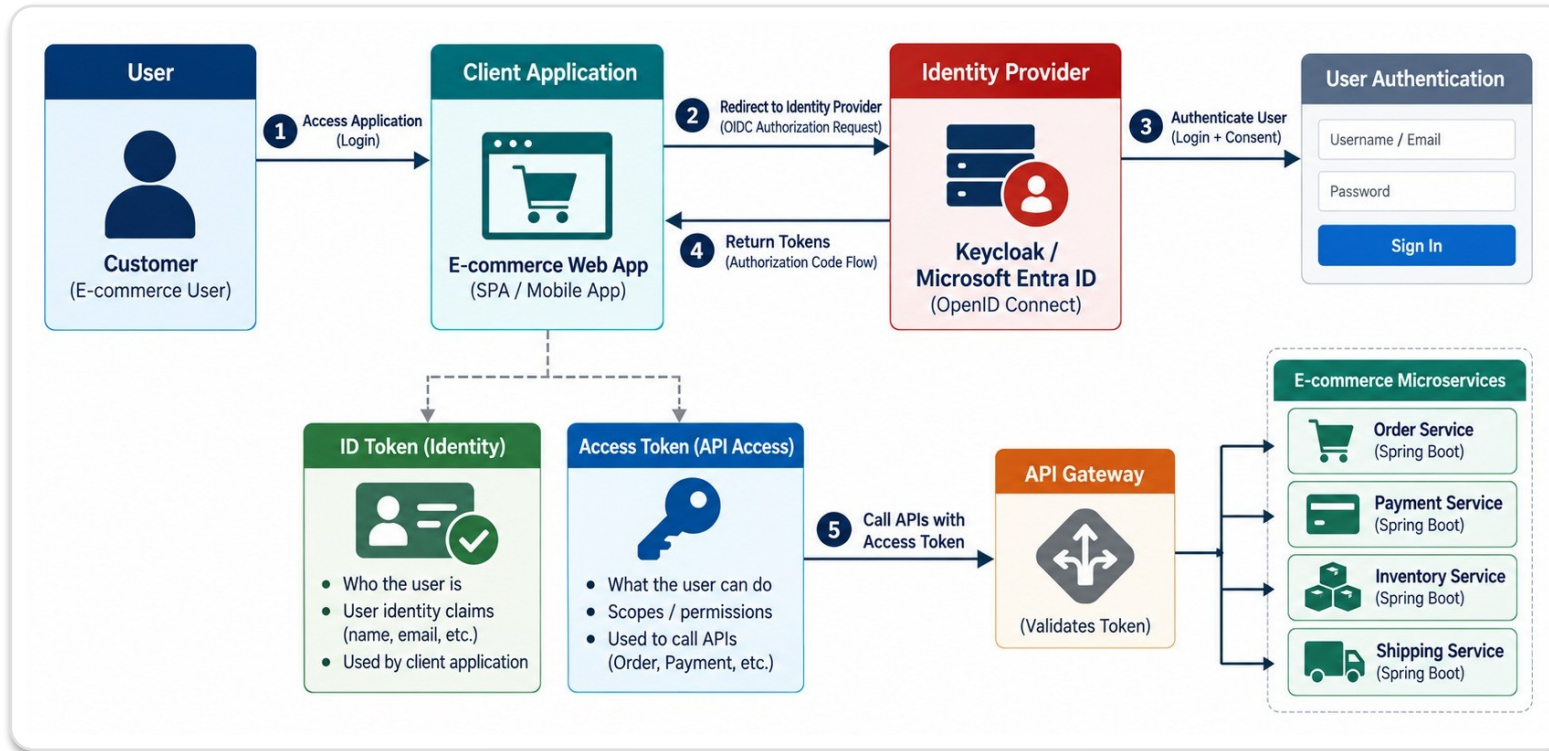
Authorization server
 Issues tokens — Keycloak or Entra ID.

Resource server
 Your API, which validates the token.

KEY TAKEAWAY OAuth 2.0 grants limited access with tokens and scopes — never shared passwords.

OPENID CONNECT FUNDAMENTALS

The authentication layer, ID token and user identity.



ID token

Who signed in — read by the client app.

Access token

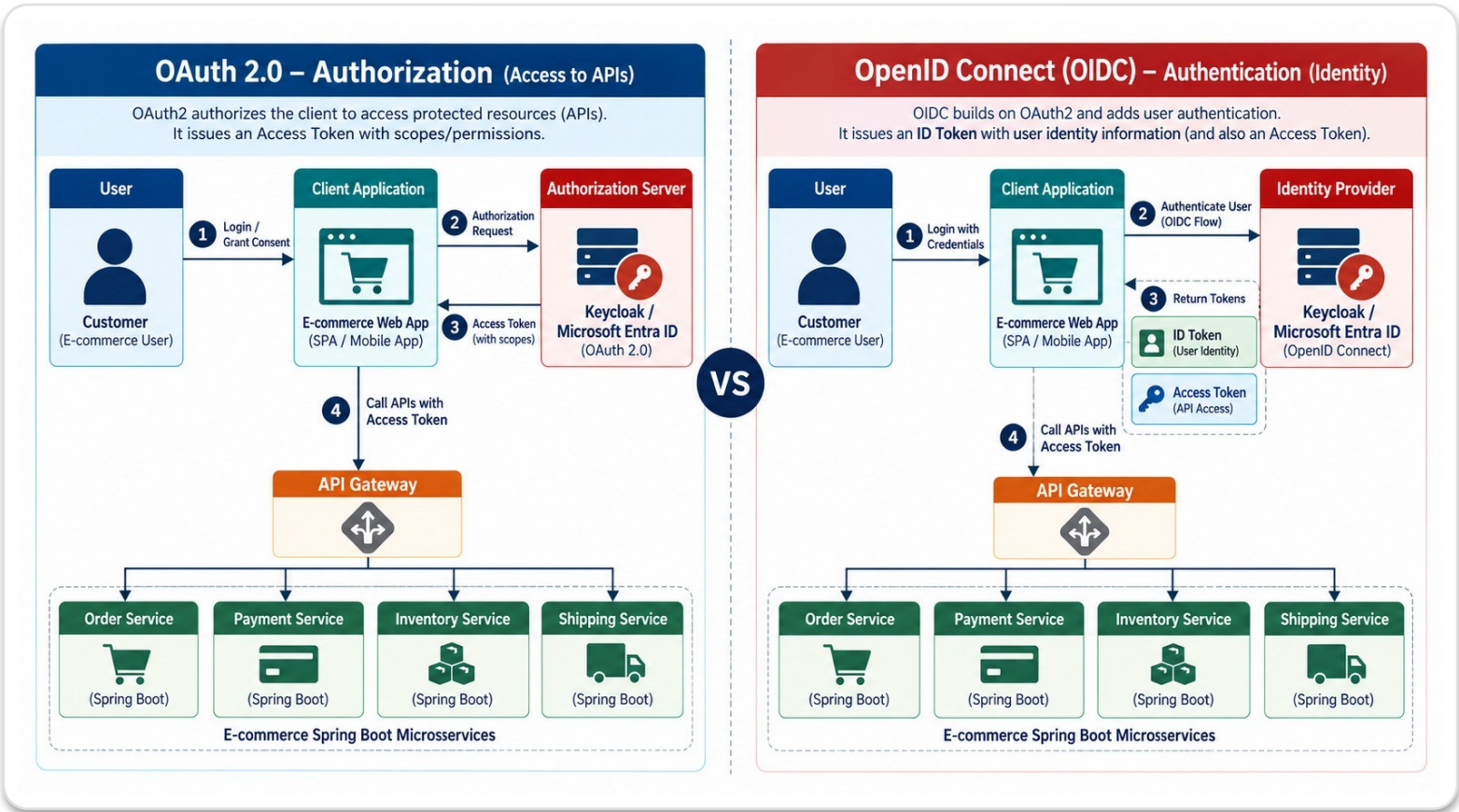
What the client may access — sent to APIs.

RULE Never send an ID token to an API as authorization.

KEY TAKEAWAY OIDC adds identity on top of OAuth 2.0 — ID tokens for clients, access tokens for APIs.

OAUTH2 VS. OPENID CONNECT

Authorization versus authentication.



OAuth 2.0 answers

What may this client access?

OIDC answers

Who is the signed-in user?

WHAT OIDC ADDS

openid scope

ID token

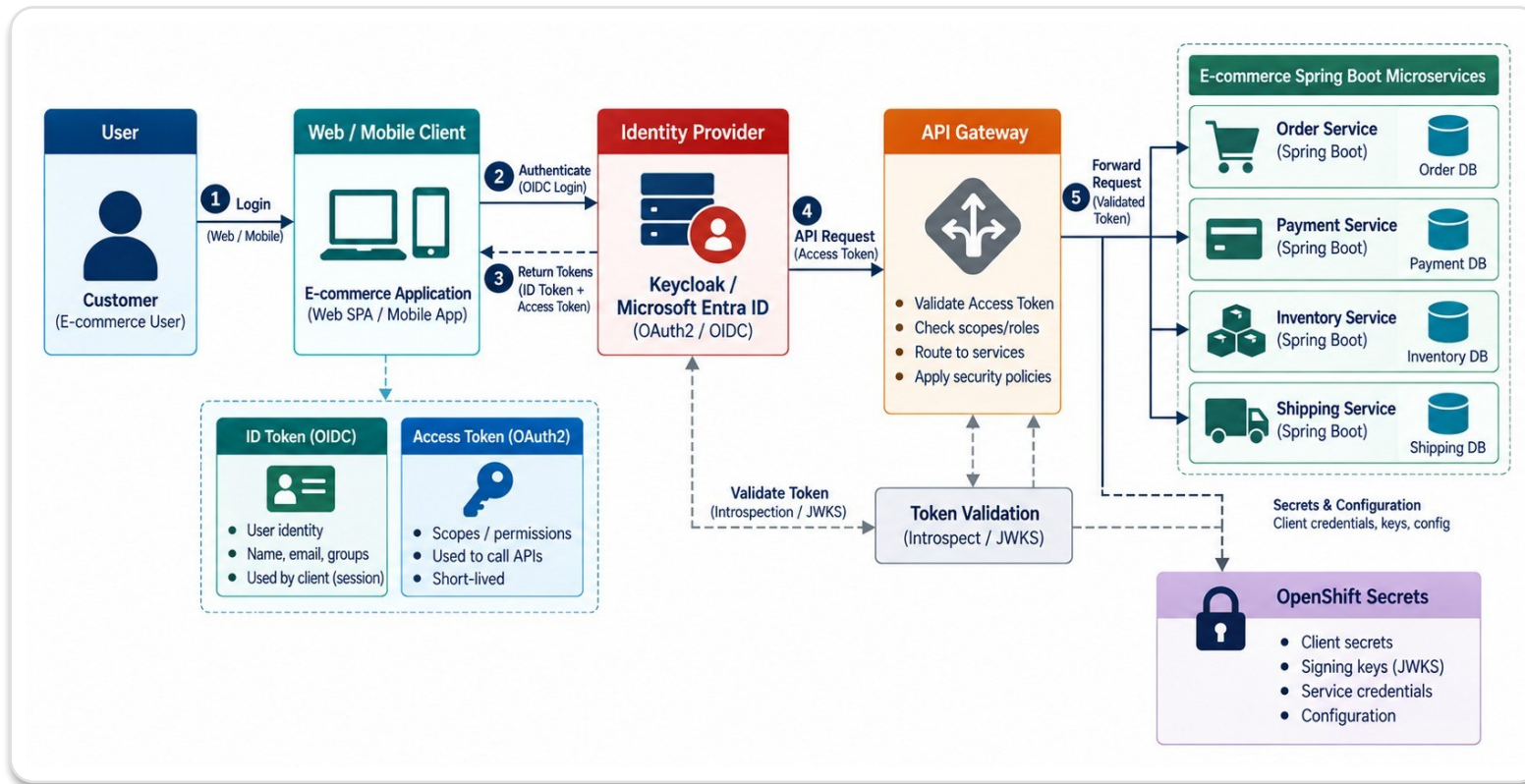
UserInfo

Discovery

KEY TAKEAWAY OIDC answers who signed in; OAuth 2.0 answers what the client may access.

OAuth2/OIDC ARCHITECTURE FOR MICROSERVICES

User → identity provider → token → API gateway → services.



AVOID

- ✗ Services storing passwords
- ✗ Only the gateway checking tokens
- ✗ Internal calls with no identity

Better

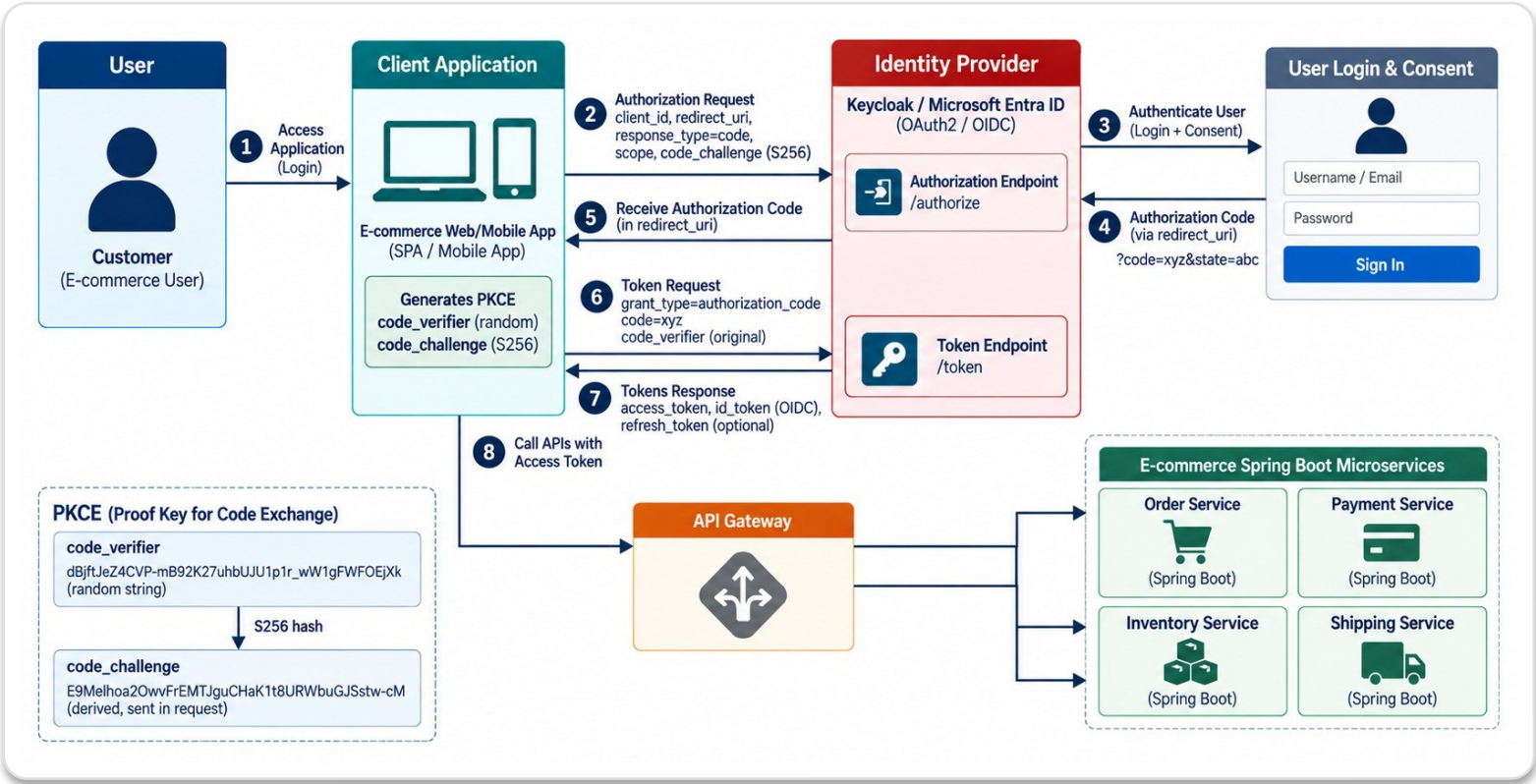
The gateway validates and routes; each service still authorizes.

STATELESS Services validate tokens without calling the IdP.

KEY TAKEAWAY Clients get tokens from the identity provider; every service still enforces authorization.

AUTHORIZATION CODE FLOW WITH PKCE

The recommended interactive user authentication flow.



Verifier

The client makes a random secret and sends only its hash.

Exchange

Code + verifier → token; the server checks they match.

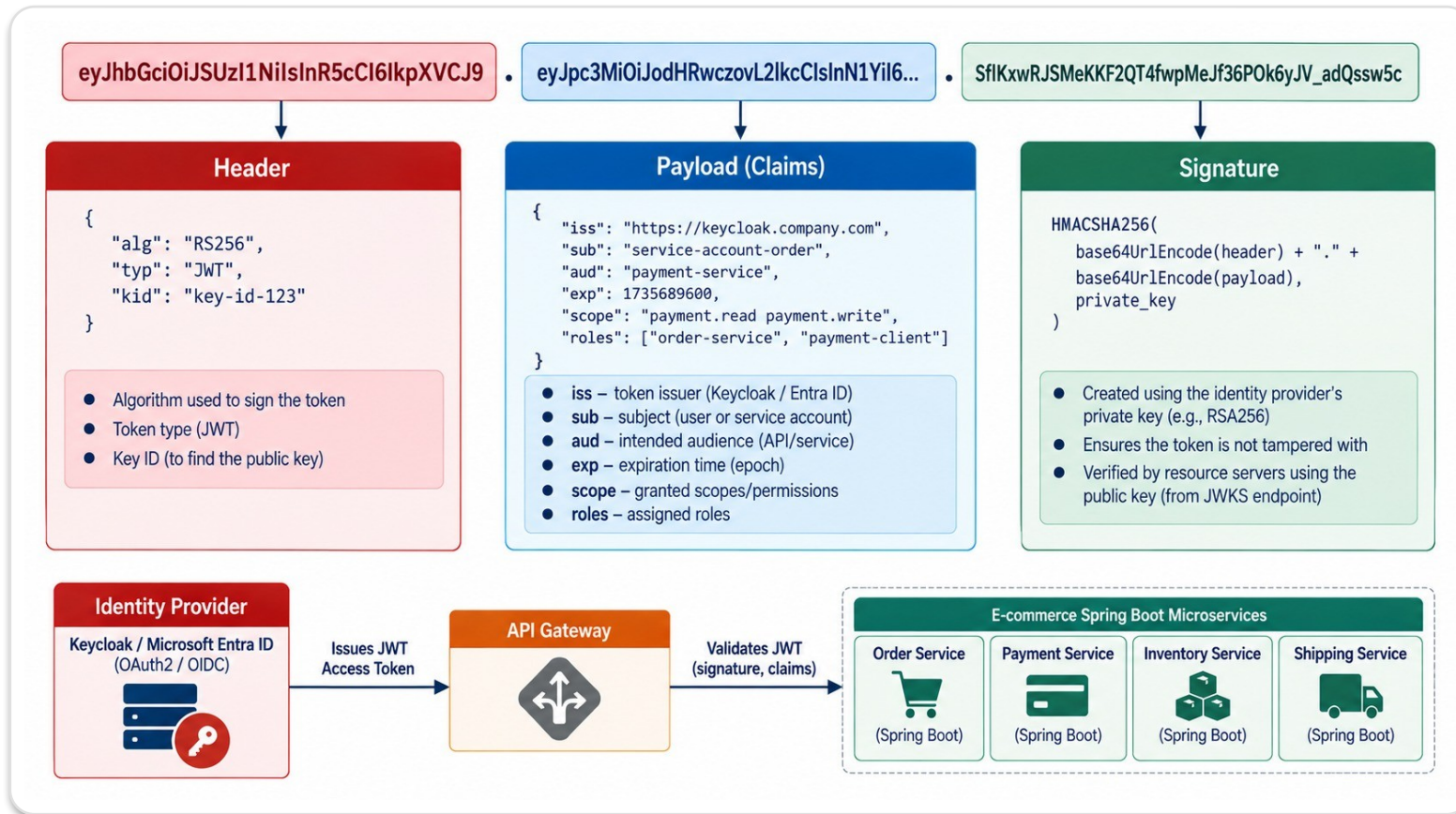
DON'T USE

- ✗ Implicit flow
- ✗ Password grant

KEY TAKEAWAY Authorization Code + PKCE protects the code exchange; stolen codes alone are useless.

UNDERSTANDING JWT ACCESS TOKENS

Header, payload, signature and claims.



A decoded payload

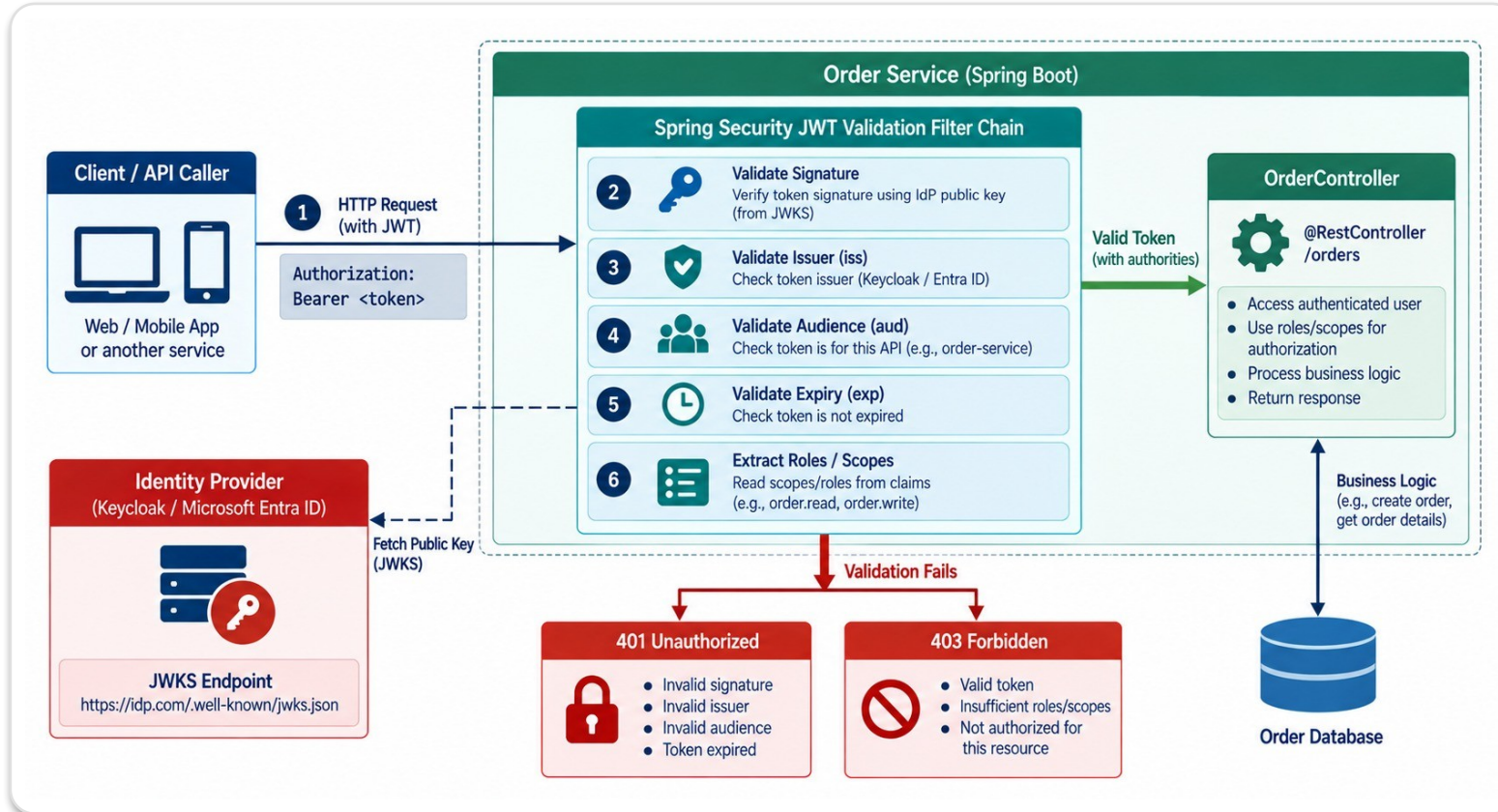
```
{
  "iss": "https://idp.example.com/realms/md287",
  "sub": "CUST-0007",
  "aud": "account-service",
  "exp": 1789472400,
  "scope": "accounts.read",
  "roles": ["teller"]
}
```

ENCODED ≠ ENCRYPTED Anyone can read the claims — keep secrets out.

KEY TAKEAWAY A JWT is signed and encoded — not encrypted — so keep secrets out of its claims.

VALIDATING TOKENS IN SPRING BOOT

Resource-server validation: signature, issuer, audience and expiry.



SPRING CHECKS

- ✓ Signature from the issuer's keys
- ✓ iss matches the issuer
- ✓ aud includes this API
- ✓ exp not passed

Then

Scopes and roles decide 403 or the controller.

KEYS Fetched from the issuer's JWKS endpoint and cached.

KEY TAKEAWAY Validate signature, issuer, audience, expiry and permissions before trusting any token.

A SPRING BOOT RESOURCE SERVER

Let Spring Security validate every JWT, then map endpoints to scopes.

application.yml

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.example.com/realms/md287
          audiences: account-service

# Spring fetches the signing keys from the
# issuer's discovery document and JWKS
```

issuer-uri

Signature, issuer and expiry are checked automatically.

audiences

Tokens minted for other APIs are rejected.

Scopes

The scope claim becomes SCOPE_ authorities.

✓ SecurityConfig.java

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .csrf(c -> c.disable()) // stateless bearer API
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health/**").permitAll()
            .requestMatchers(GET, "/api/v1/accounts/**")
                .hasAuthority("SCOPE_accounts.read")
            .requestMatchers(POST, "/api/v1/accounts/**")
                .hasAuthority("SCOPE_accounts.write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

KEY TAKEAWAY Configure the issuer and audience, then map each endpoint to the narrowest scope.

OBJECT-LEVEL AUTHORIZATION

A valid scope isn't enough — check this caller may touch this account.

✗ Only checks the scope

```
@GetMapping("/{accountId}")
@PreAuthorize("hasAuthority('SCOPE_accounts.read')")
AccountResponse get(@PathVariable String accountId) {
    return accounts.find(accountId);
}

// any customer can read ANY account
// by changing the id in the URL
```

Broken object-level authorization

The top API security risk.

✓ Checks ownership too

```
@GetMapping("/{accountId}")
@PreAuthorize("hasAuthority('SCOPE_accounts.read')")
AccountResponse get(@PathVariable String accountId,
    @AuthenticationPrincipal Jwt jwt) {
    Account account = accounts.find(accountId);
    boolean staff = jwt.getClaimAsStringList("roles")
        .contains("teller");
    if (!staff && !account.customerId().equals(jwt.getSubject()))
        throw new AccessDeniedException("Not the account owner");
    return AccountResponse.from(account);
}
```

Ownership

Compare the token subject with the resource owner.

Staff roles

Allowed by role — and audited.

KEY TAKEAWAY Authorize the object, not just the endpoint — compare who's asking with who owns it.

EXERCISE 3.2 — SECURE AN ENDPOINT WITH JWT ROLES/SCOPES

Day 3 checkpoint B: roles and scopes on protected APIs.

OFFICIAL CHECKPOINT B

10 MIN

labs/day-03/exercises/exercise-3.2-jwt-roles-scopes.md

SCENARIO

Lab 3 secures the banking Account and Transaction services with JWTs issued by `issue-jwt.py` (HMAC). Keycloak and Entra ID stay lecture-only.

YOUR TASKS

- 1 No Bearer token → 401; wrong scope → 403 — not a silent 200
- 2 teller: accounts.read/write; ops: transactions + accounts.read; readonly: *.read
- 3 Transaction Service forwards the caller's JWT when it GETs Account Service
- 4 Never log Authorization headers or JWT claims

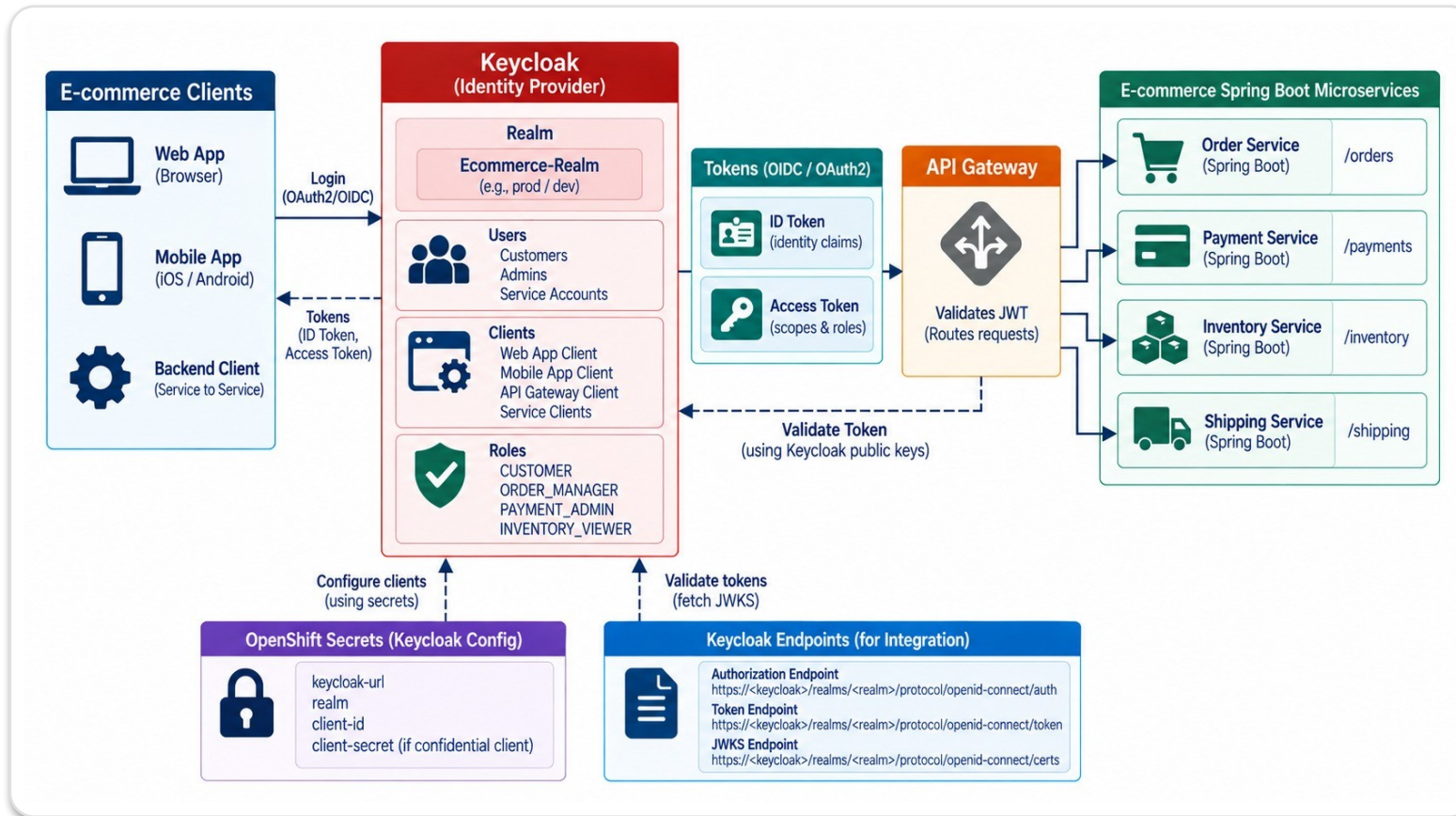
EXPECTED OUTCOME

- ✓ A scope matrix per role
- ✓ Testable with `issue-jwt.py` teller / ops / readonly
- ✓ 401 and 403 in the right places
- ✓ No tokens or claims in logs

KEY TAKEAWAY A scope matrix you can test with `issue-jwt.py` teller, ops and readonly.

KEYCLOAK ARCHITECTURE AND INTEGRATION

Realm, clients, users, roles, tokens and a Spring Boot resource server.



Realm

An isolated security domain — for example md287.

Client

Each app or service registered with the realm.

Roles

Realm and client roles mapped into token claims.

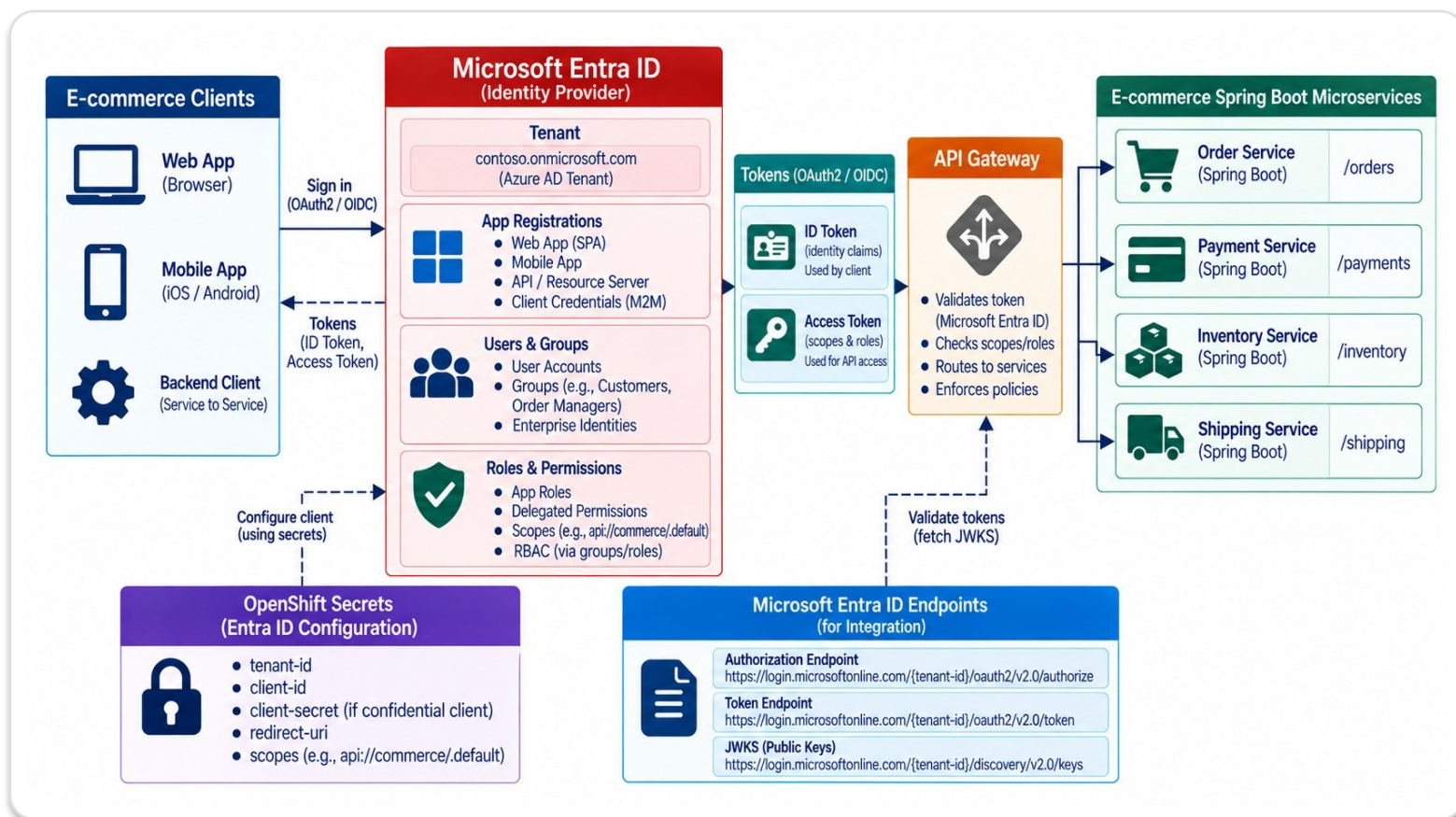
Spring points at the realm

issuer-uri:
<https://keycloak.example.com/realms/md287>

KEY TAKEAWAY Keycloak realms, clients, users and roles become standard OIDC tokens for Spring Boot.

MICROSOFT ENTRA ID (FORMERLY AZURE AD)

App registration, identity platform, tokens and Spring Boot APIs.



App registration

Defines the API, its scopes and app roles.

Tokens

Standard OAuth2 and OIDC — with tenant and app-role claims.

Same Spring configuration

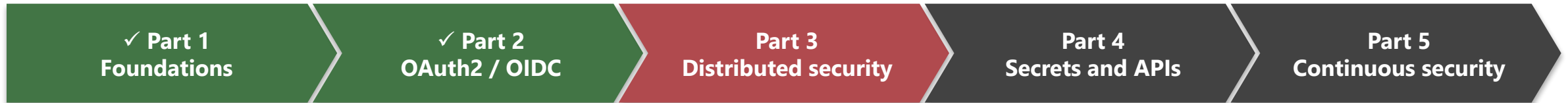
```
issuer-uri:  
https://login.microsoftonline.com/<tenant>/v2  
.0
```

STAY PORTABLE Code against OAuth2, not one vendor.

KEY TAKEAWAY Microsoft Entra ID issues standard OAuth2/OIDC tokens that Spring Boot validates.

PART 3: DISTRIBUTED SECURITY

Users aren't the only identities — services, agents and pipelines are too.



SO FAR

- ✓ OAuth2 issues tokens; OIDC adds identity
- ✓ Spring validates issuer, audience and expiry
- ✓ Services check scopes and ownership

NEXT QUESTION

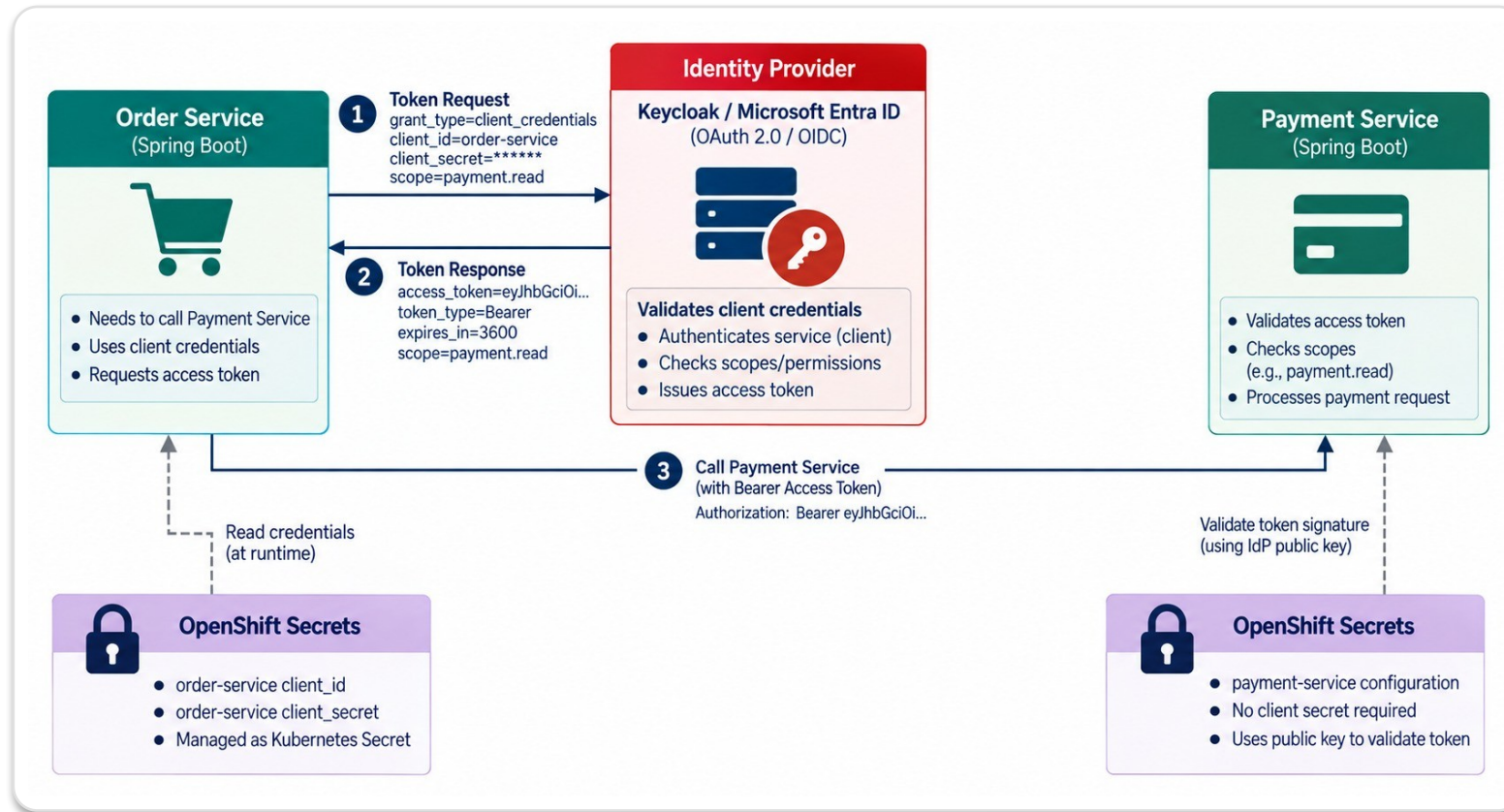
How do services, gateways and agents prove who they are to each other?

IN THIS PART

- 1 Client credentials — in code
- 2 API gateway security
- 3 Service-to-service authentication

CLIENT CREDENTIALS FLOW FOR SERVICE-TO-SERVICE SECURITY

Machine identities for microservice communication.



A machine acting as itself

Order Service authenticates with its own client ID, not a user's.

Least privilege

It gets payments.authorize — not payments.admin.

AUDIT Logs show which service made each call.

KEY TAKEAWAY Services authenticate as themselves with narrowly scoped tokens.

SERVICE TOKENS WITH CLIENT CREDENTIALS

Spring fetches, caches and attaches a service token to every outgoing call.

application.yml

```
spring:
  security:
    oauth2:
      client:
        registration:
          order-service:
            provider: idp
            client-id: order-service
            client-secret: ${ORDER_CLIENT_SECRET}
            authorization-grant-type: client_credentials
            scope: payments.authorize
        provider:
          idp:
            issuer-uri: https://idp.example.com/realms/md287
```

✓ PaymentClientConfig.java

```
@Bean
RestClient paymentClient(RestClient.Builder builder,
                        OAuth2AuthorizedClientManager clients) {
    var oauth = new OAuth2ClientHttpRequestInterceptor(clients);
    oauth.setClientRegistrationIdResolver(req -> "order-service");
    return builder
        .baseUrl("http://payment-service:8080")
        .requestInterceptor(oauth) // adds Bearer token
        .build();
}

// Payment Service requires SCOPE_payments.authorize
```

Secret

From an OpenShift Secret — never in the file.

Caching

Tokens are reused until they expire.

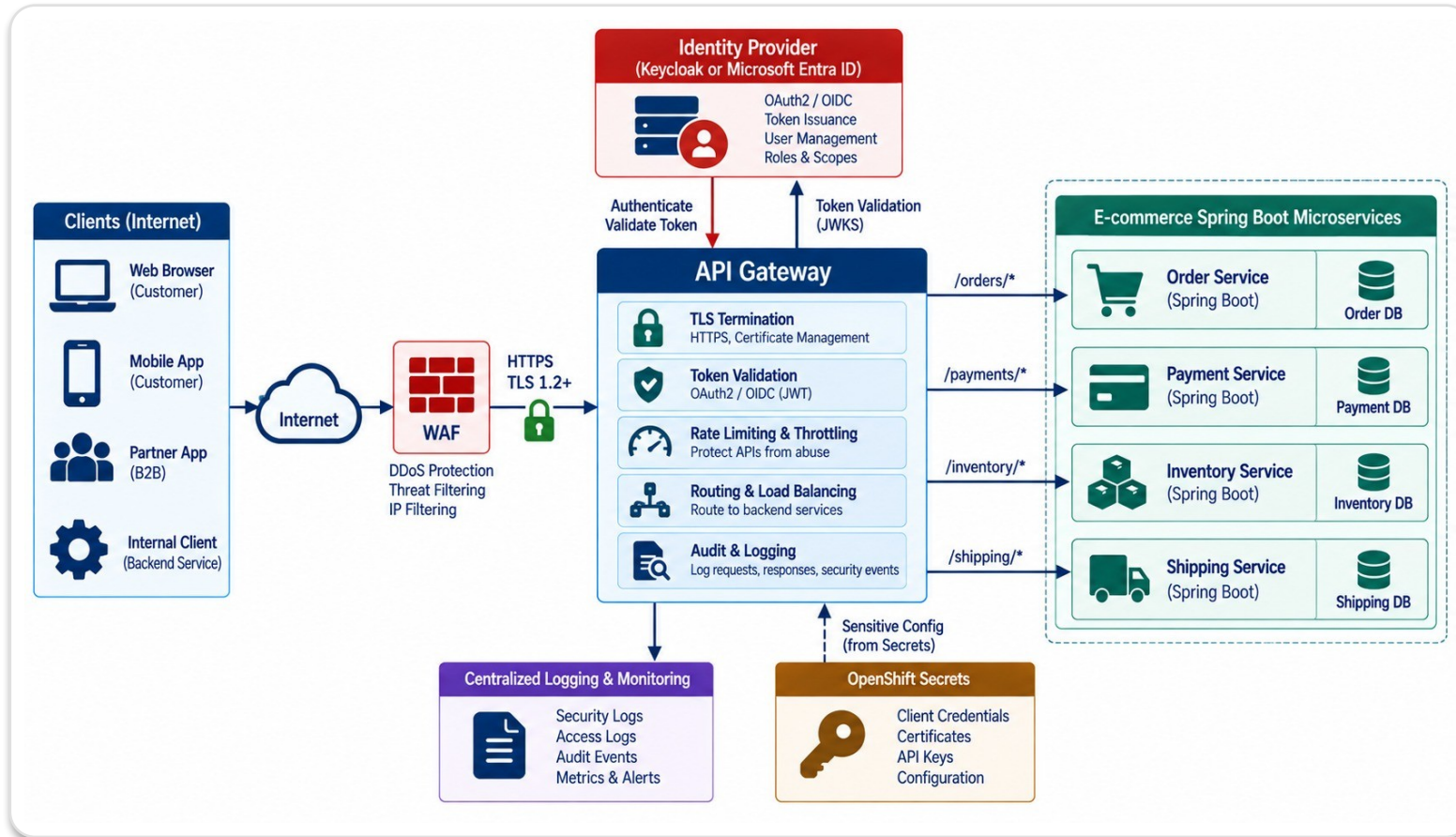
Lab 3 instead

Forwards the caller's JWT to keep user context.

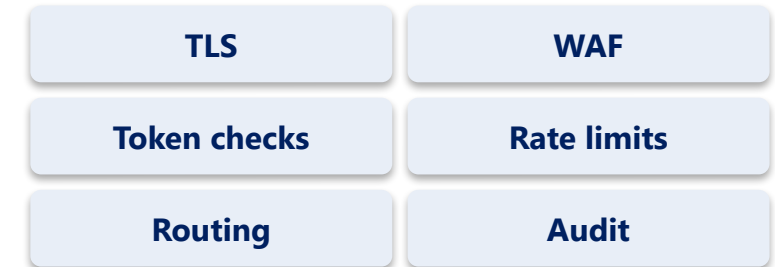
KEY TAKEAWAY Give each service its own client, narrow scopes and automatic token handling.

API GATEWAY SECURITY

Central protection, routing, throttling and policy — but not the only layer.



AT THE EDGE



NOT AT THE GATEWAY

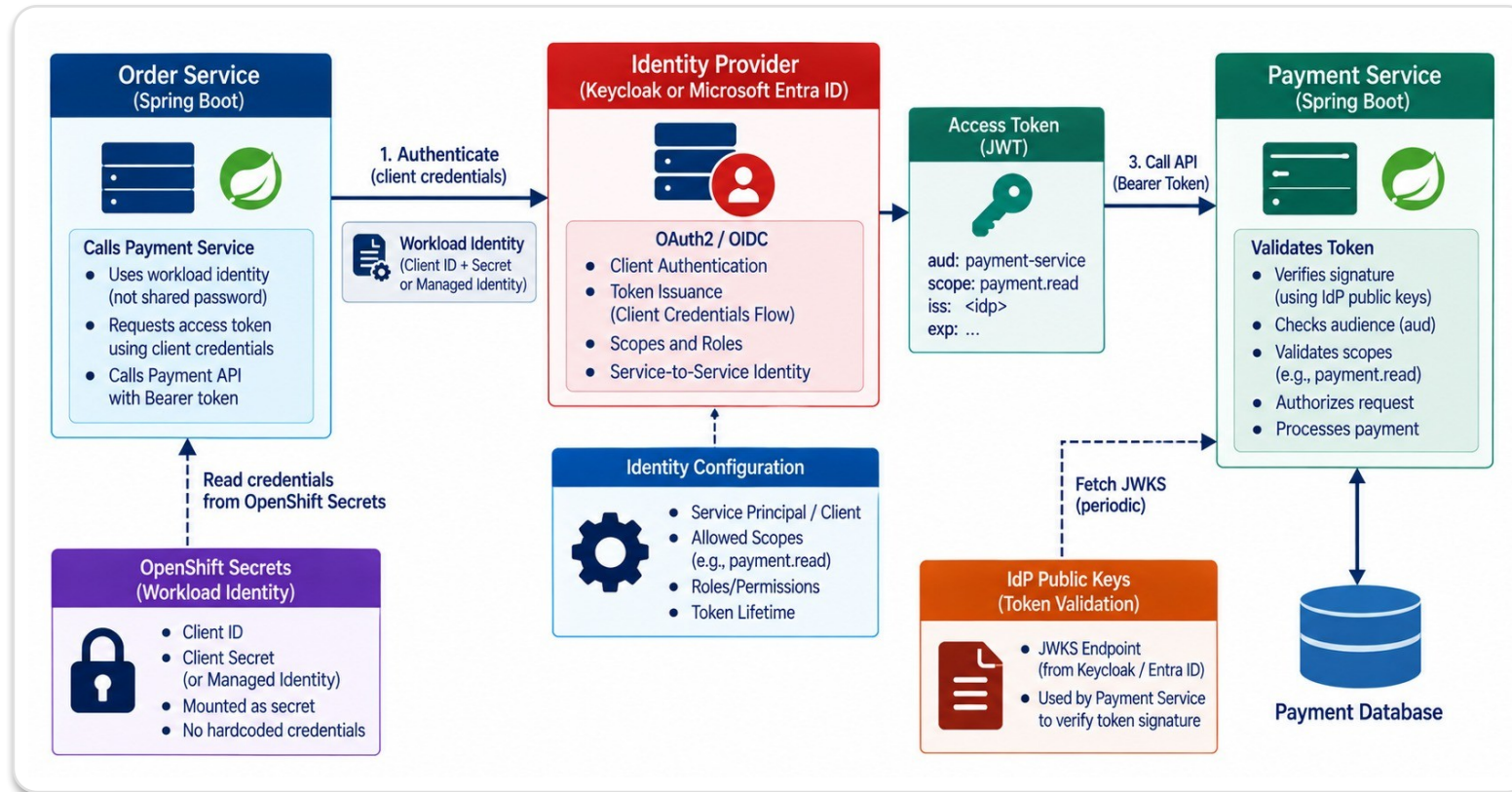
- ✗ Object ownership
- ✗ Business rules

WHY Internal callers can bypass the gateway.

KEY TAKEAWAY Protect the edge at the gateway and still enforce authorization in every service.

SERVICE-TO-SERVICE AUTHENTICATION

Machine identity, bearer tokens and workload credentials.



Workload identity

The platform gives each pod an identity — no long-lived secret to leak.

mTLS

A service mesh can authenticate both ends of every call.

IDENTITIES TO SECURE

Services

Agents

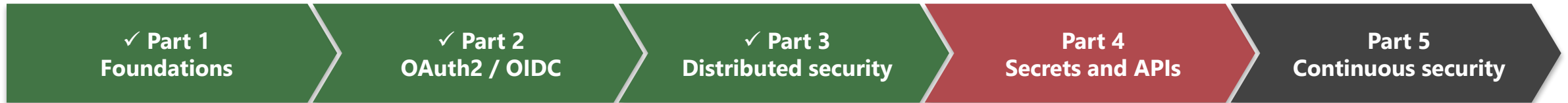
MCP servers

CI/CD jobs

KEY TAKEAWAY Every workload authenticates with its own identity and least-privilege scopes.

PART 4: SECRETS AND API PROTECTION

Keep credentials out of code — and harden every endpoint.



SO FAR

- ✓ Services have their own identities
- ✓ Gateways protect the edge
- ✓ Services still authorize every call

NEXT QUESTION

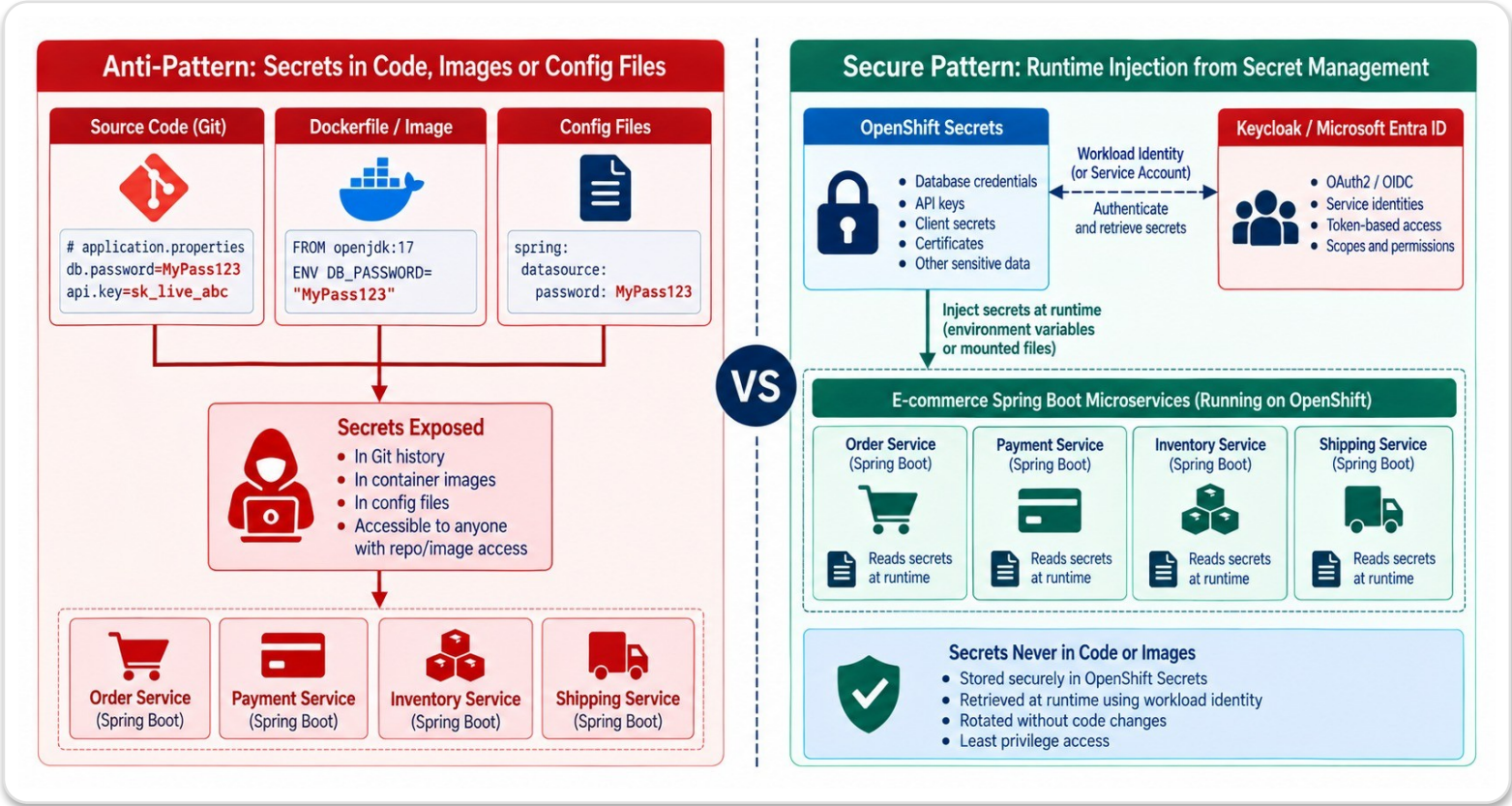
Where do credentials live — and what else protects our APIs?

IN THIS PART

- 1 Secrets management fundamentals
- 2 Secrets in OpenShift — in code
- 3 Exercise: find the secret leaks
- 4 API security best practices
- 5 Common API threats — with an exercise

SECRETS MANAGEMENT FUNDAMENTALS

Why credentials don't belong in code, images or config repositories.



WHAT COUNTS AS A SECRET

DB passwords

Client secrets

API keys

Signing keys

TLS private keys

Images remember

A secret in any layer stays in the image history.

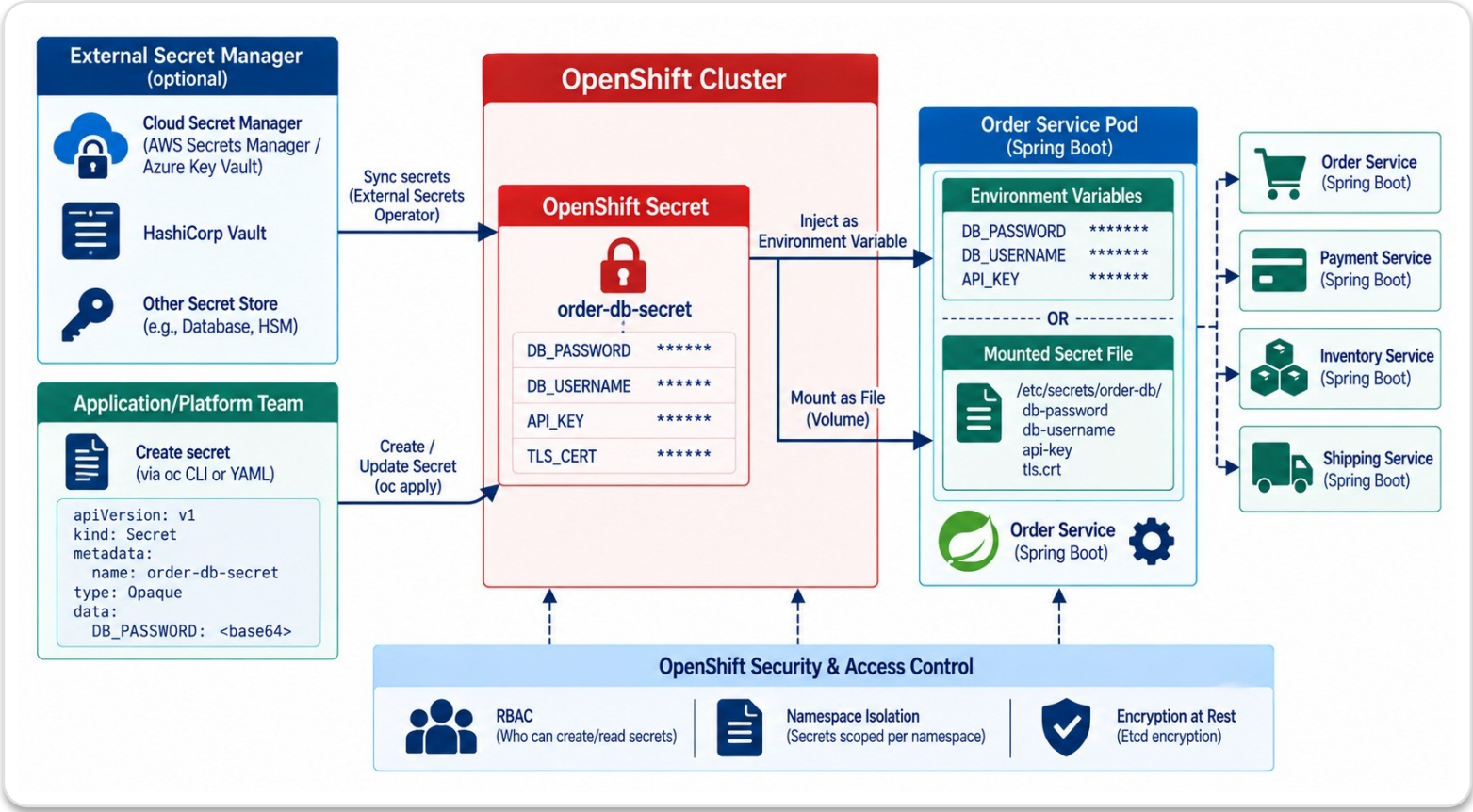
LIFECYCLE

Store, restrict, rotate, audit, revoke.

KEY TAKEAWAY Secrets need stronger storage, access, rotation and audit than normal configuration.

SECRETS MANAGEMENT IN OPENSIFT

Secret injection, environment variables and files, external secret stores.



Delivery, not strategy

OpenShift Secrets deliver values; a vault manages and rotates them.

EXTERNAL MANAGERS

HashiCorp Vault

Azure Key Vault

External Secrets

CyberArk

RBAC Only the service account may read its Secret.

KEY TAKEAWAY Inject secrets at runtime with tight access — never bake them into the image.

MOUNTING SECRETS AS FILES

Files are harder to leak than environment variables — and Spring reads them directly.

Deployment (excerpt)

```
spec:
  template:
    spec:
      serviceAccountName: account-service
      volumes:
        - name: db-credentials
          secret:
            secretName: account-db-secret
      containers:
        - name: account-service
          volumeMounts:
            - name: db-credentials
              mountPath: /etc/secrets
              readOnly: true
```

✓ application.yml

```
spring:
  config:
    import: "optional:configtree:/etc/secrets/"

# Secret keys become files, file names become
# properties:
#   /etc/secrets/spring.datasource.password
#   -> spring.datasource.password

# never:
#   log.info("config: {}", dataSourceProperties);
```

Read-only mount

The pod can read, not modify, its secrets.

configtree

Spring maps each file to a property.

Rotation

Updated Secret files appear without rebuilding the image.

KEY TAKEAWAY Mount secrets read-only at runtime and let Spring read them — never bake or log them.

EXERCISE: FIND THE SECRET LEAKS

Review a release for places a credential could escape.

PRACTICE EXERCISE

10 MIN

SCENARIO

A release of Payment Service contains:

```
application-prod.yml:
  gateway.api-key: sk_live_9f2c...
Dockerfile:
  ENV DB_PASSWORD=Payments#2026
PaymentService.java:
  log.info("Calling gateway with {}", headers)
ci.yml:
  run: echo $REGISTRY_TOKEN
```

YOUR TASKS

- 1 Find four leaks
- 2 Say who could see each secret
- 3 Fix each one — and decide what to rotate now

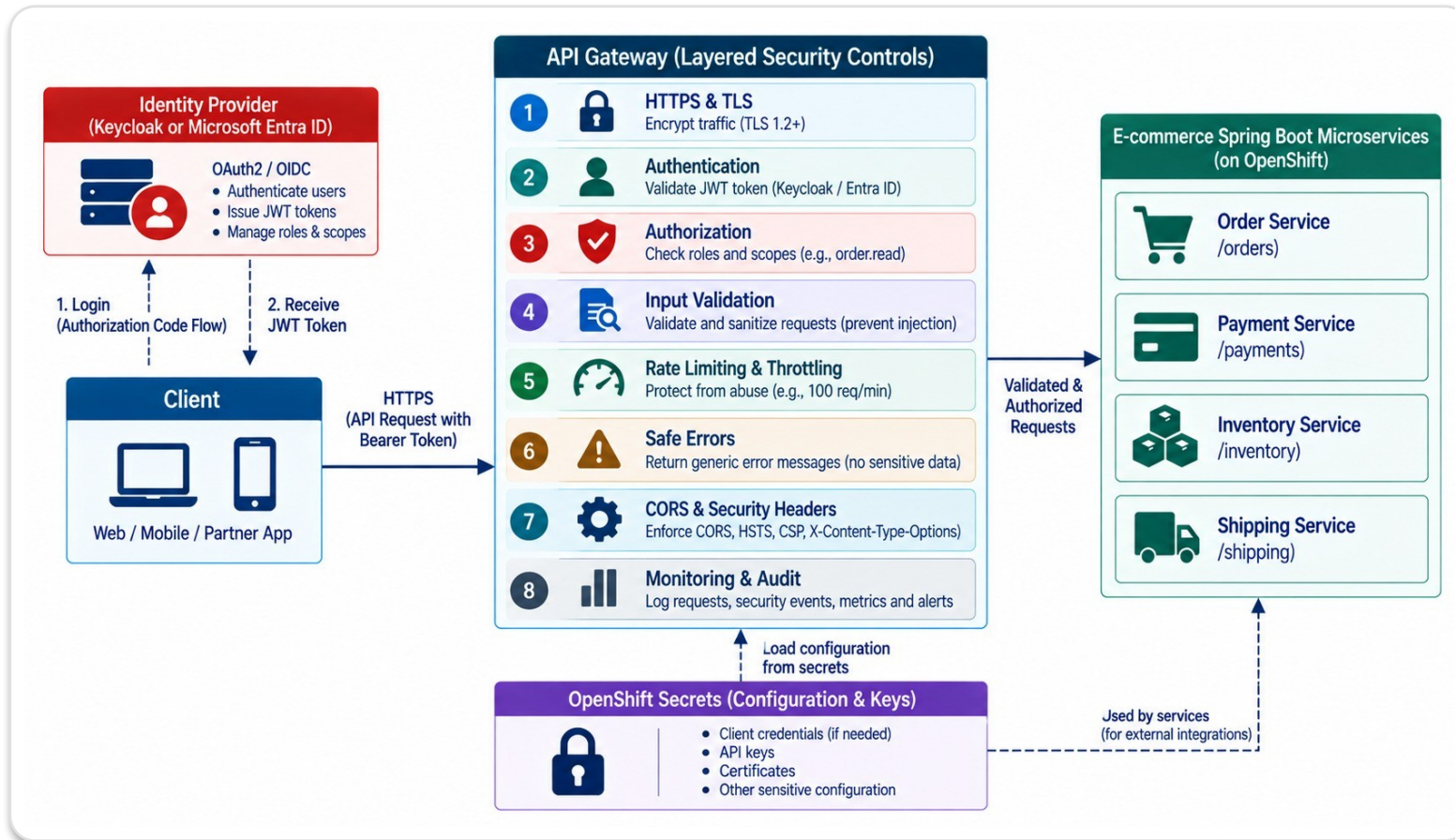
EXPECTED OUTCOME

- ✓ API key in Git → secret store + rotate
- ✓ Password in image layer → runtime Secret
- ✓ Headers logged → mask Authorization
- ✓ Token echoed in CI → masked secrets
- ✓ Rotate everything already exposed

KEY TAKEAWAY Anything committed, baked, logged or echoed is exposed — move it to a store, and rotate it.

API SECURITY BEST PRACTICES

HTTPS, input validation, authorization, CORS, rate limiting and safe errors.



✓ Safe error

```
{ "error": "TRANSFER_REJECTED",  
  "message": "The transfer could not be  
completed." }
```

NEVER RETURN

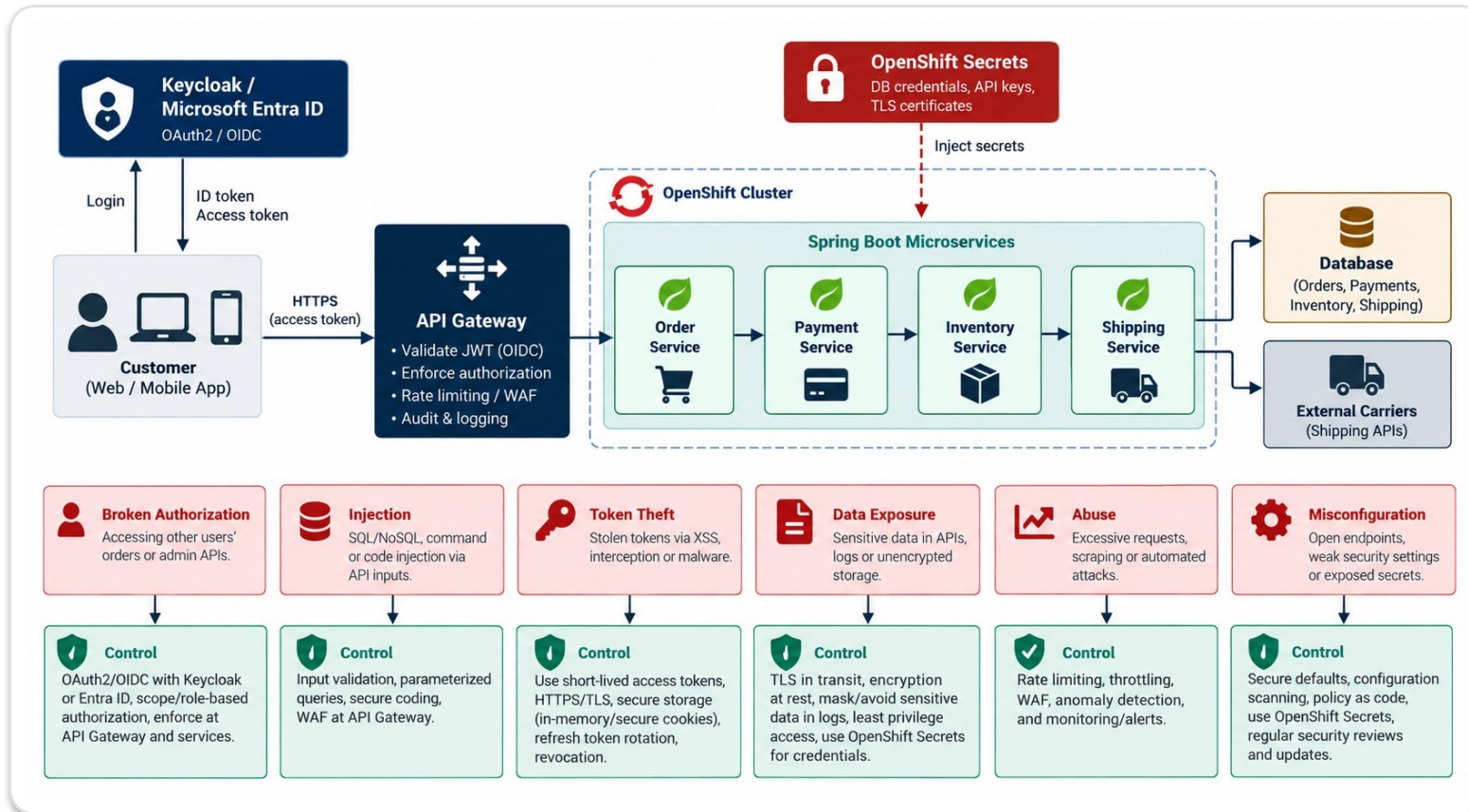
- ✗ Stack traces
- ✗ SQL details
- ✗ Internal hostnames

MONEY APIS need an Idempotency-Key.

KEY TAKEAWAY Layer the controls: encrypt, authenticate, authorize, validate, limit, respond safely, monitor.

COMMON API SECURITY THREATS

Broken authorization, injection, token leakage, data exposure and abuse.



Broken object authorization

→ check ownership in the service.

Excessive data exposure

→ DTOs with only needed, masked fields.

Token theft

→ short lifetimes, HTTPS, no tokens in logs.

REFUNDS need step-up checks and audit.

KEY TAKEAWAY A valid token isn't enough — authorize each object and action, and expose only what's needed.

EXERCISE: MAP THREATS TO CONTROLS

Match each finding from a security review to its fix.

PRACTICE EXERCISE

10 MIN

SCENARIO

Findings from a review of the Transfer API:

- 1 Changing accountId shows others' data
- 2 Response includes full card number
- 3 No limit on POST /transfers
- 4 Error returns the SQL statement
- 5 Retrying a timeout sends money twice
- 6 CORS allows any origin

YOUR TASKS

- 1 Name the threat behind each finding
- 2 Choose the control that fixes it
- 3 Say where the control belongs

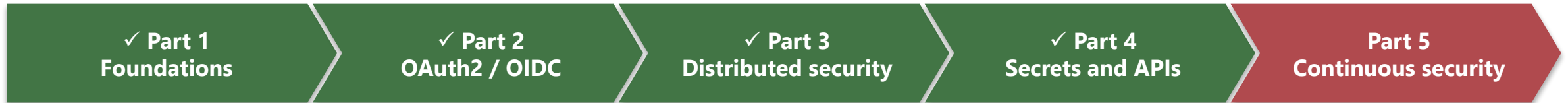
EXPECTED OUTCOME

- ✓ 1 Broken object authorization → service check
- ✓ 2 Data exposure → masked DTO
- ✓ 3 Abuse → rate limit at the gateway
- ✓ 4 Info leak → safe error handler
- ✓ 5 Replay → Idempotency-Key; 6 → allow-list

KEY TAKEAWAY Each common threat has a known control — and a right place to enforce it.

PART 5: CONTINUOUS SECURITY AND COMPLIANCE

Prove security, not just claim it — and keep checking.



SO FAR

- ✓ Secrets live in stores, not code
- ✓ APIs validate, limit and fail safely
- ✓ Threats map to known controls

NEXT QUESTION

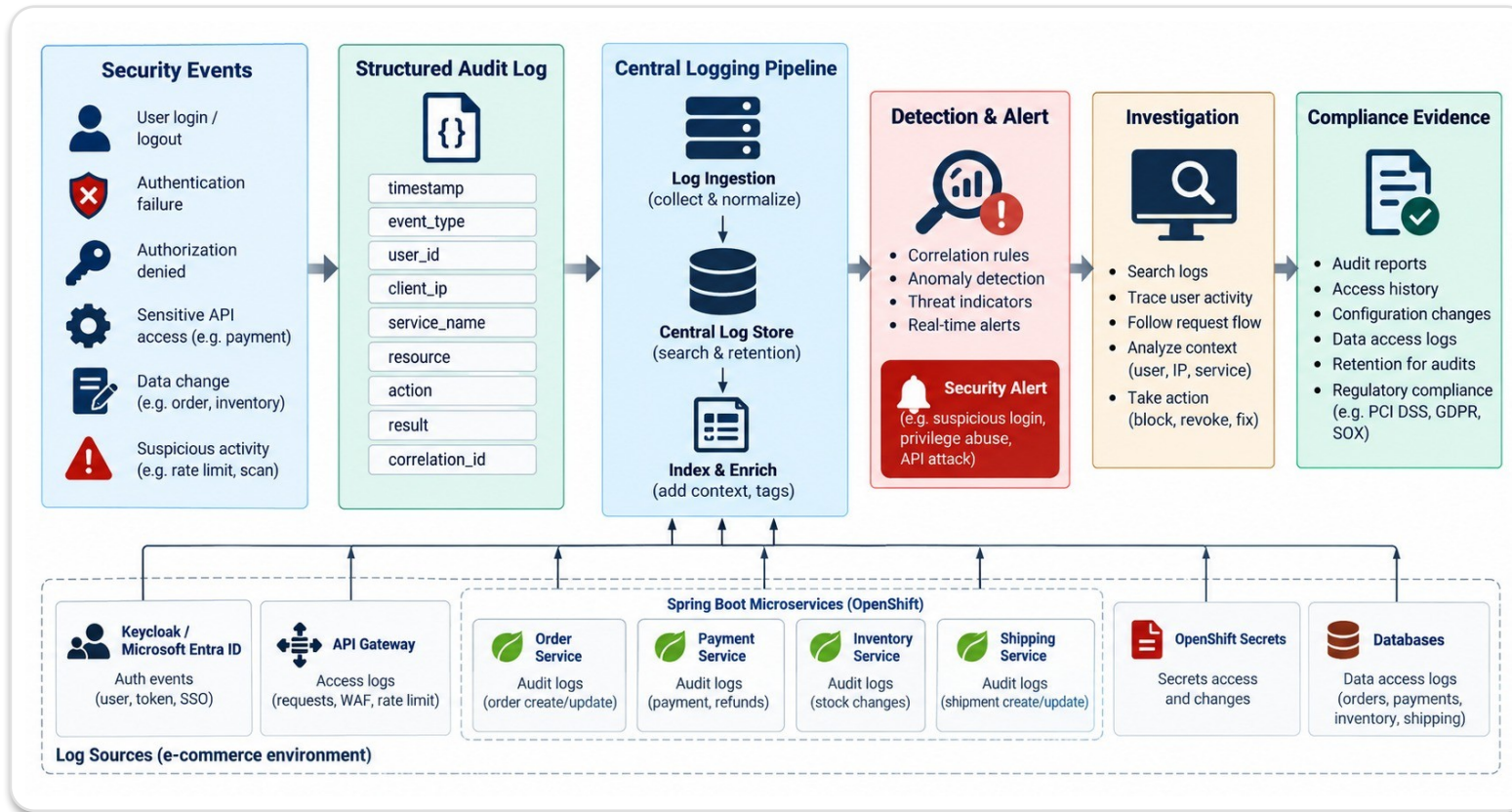
How do we prove to auditors — and ourselves — that controls work every day?

IN THIS PART

- 1 Security logging and audit — in code
- 2 DevSecOps and continuous security
- 3 The complete secure architecture

SECURITY LOGGING, AUDITING, AND COMPLIANCE

Audit events, traceability, privacy, retention and evidence.



EVERY AUDIT EVENT

- ✓ Who — subject and client
- ✓ What — action and resource
- ✓ Decision — allowed or denied
- ✓ When, and correlation ID

NEVER LOG

- ✗ Tokens, passwords
- ✗ Full card or account numbers

KEY TAKEAWAY Make security observable: record who did what, to which resource, and when.

WRITING A SECURITY AUDIT EVENT

Structured, masked, correlated — and separate from ordinary debug logs.

✓ AuditLogger.java

```
private static final Logger AUDIT = LoggerFactory.getLogger("security.audit");

public void accountFrozen(String accountId, Jwt actor, boolean allowed) {
    AUDIT.atInfo()
        .setMessage("security_audit")
        .addKeyValue("event", "ACCOUNT_FREEZE")
        .addKeyValue("actor", actor.getSubject())
        .addKeyValue("client", actor.getClaimAsString("azp"))
        .addKeyValue("accountId", mask(accountId))
        .addKeyValue("decision", allowed ? "ALLOWED" : "DENIED")
        .addKeyValue("correlationId", MDC.get("correlationId"))
        .log();
}
```

What it produces

```
{
  "logger": "security.audit",
  "event": "ACCOUNT_FREEZE",
  "actor": "ops-user-17",
  "client": "ops-console",
  "accountId": "ACC-***5001",
  "decision": "ALLOWED",
  "correlationId": "c-9a8e",
  "timestamp": "2026-09-18T14:22:11Z"
}
```

Dedicated logger

Route audit events to protected, retained storage.

Denials too

Denied attempts are often the most important.

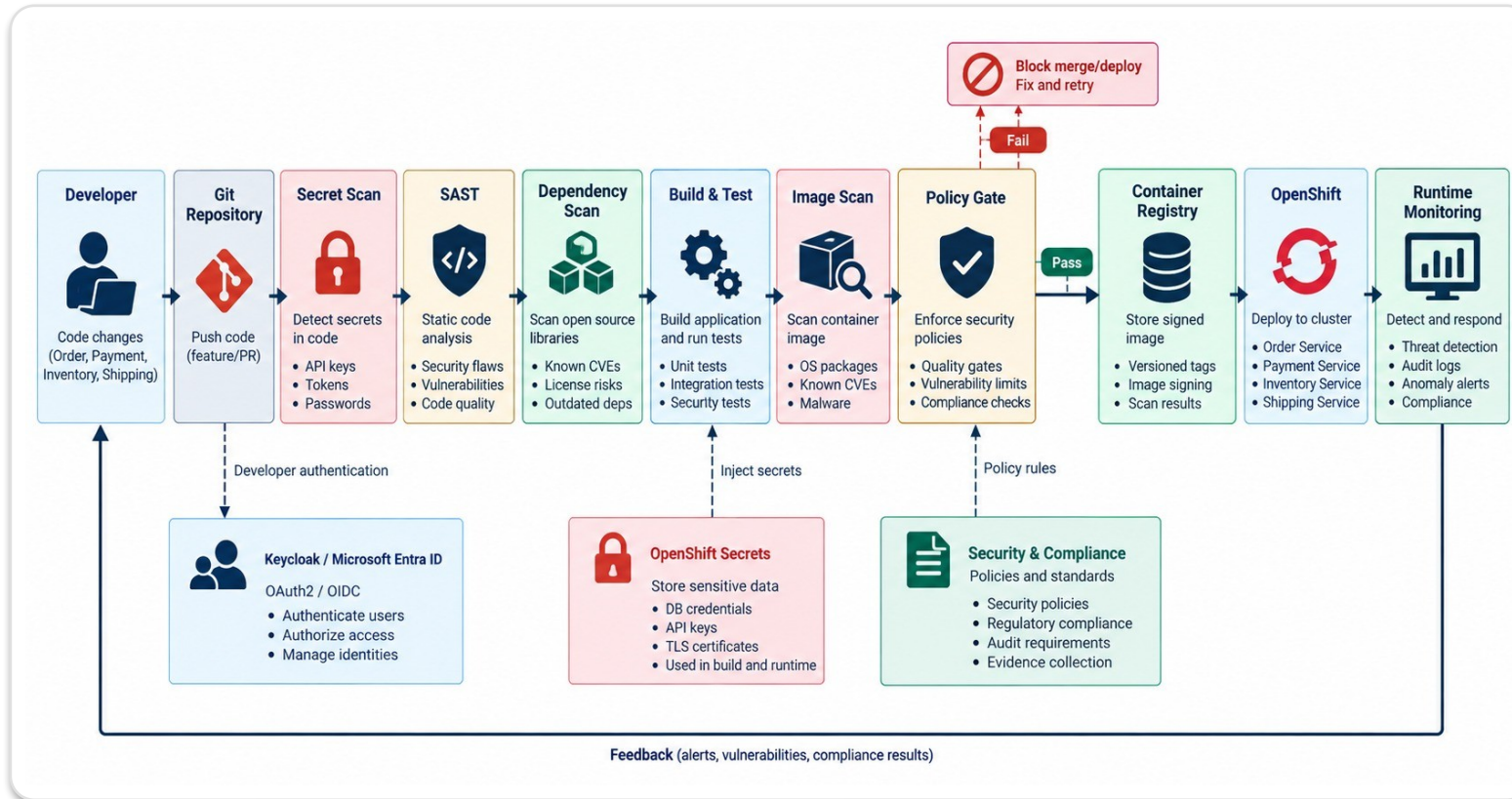
Masked IDs

Enough to investigate, not enough to misuse.

KEY TAKEAWAY Emit structured, masked audit events for every sensitive decision — allowed or denied.

DEVSECOPS AND CONTINUOUS SECURITY

SAST, dependency, secret and image scanning with policy gates.



Before merge

Secret scan and SAST on every pull request.

Before deploy

Dependency and image scans, SBOM and signature.

At runtime

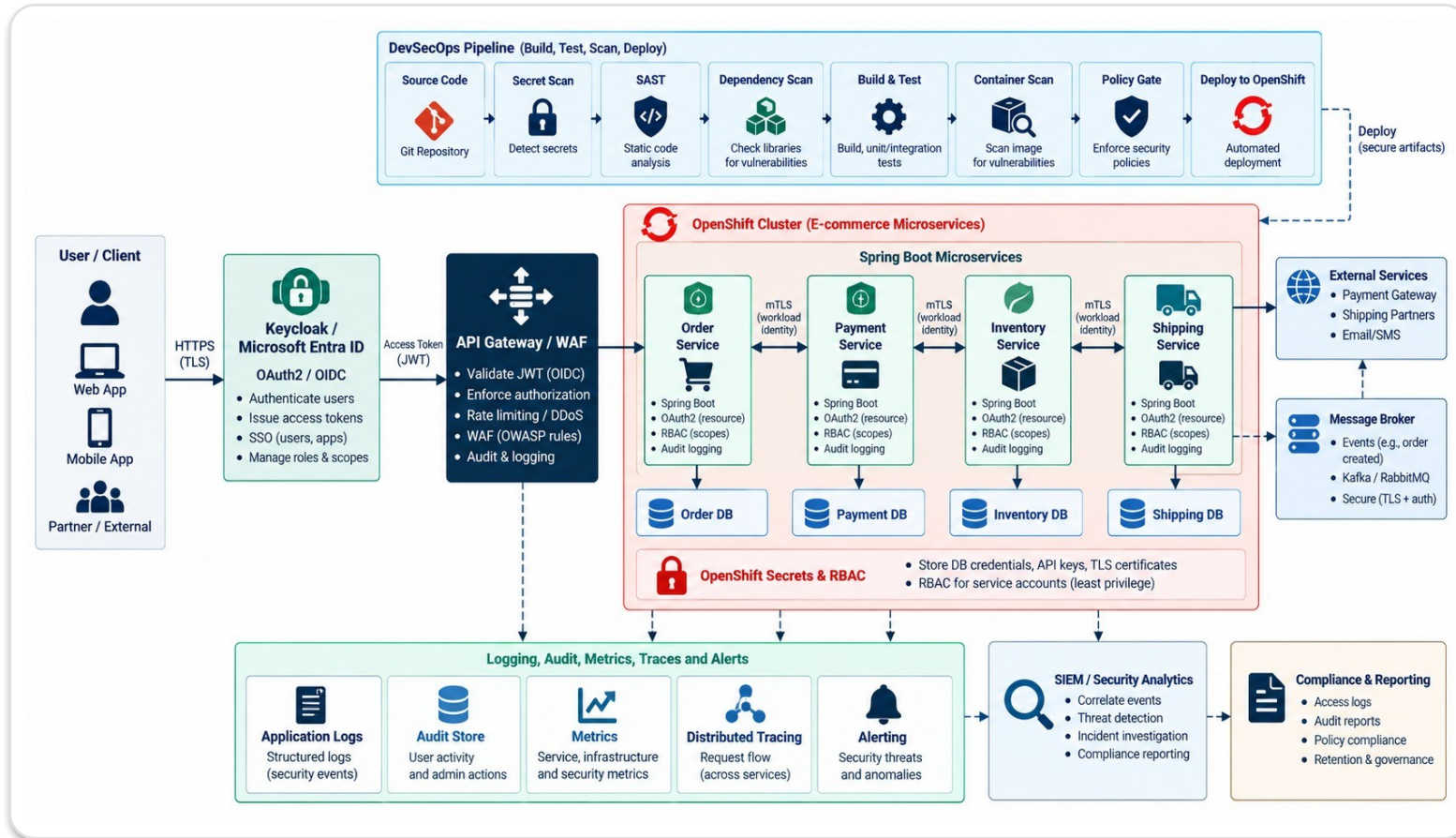
Admission policies, alerts and regular access reviews.

MODULE 7 built these gates into the pipeline.

KEY TAKEAWAY Scan code, dependencies, images and deployments continuously — the whole supply chain.

END-TO-END SECURE MICROSERVICES ARCHITECTURE

User → IdP → gateway → Spring Boot services → secrets and data → OpenShift → monitoring.



EVERY PART OF THE MODULE

- ✓ HTTPS and OIDC login with PKCE
- ✓ Gateway validates, limits and audits
- ✓ Services check scopes and ownership
- ✓ Workload identities between services
- ✓ Secrets at runtime; DevSecOps and audit

REMEMBER Trust identity and policy — never the network.

KEY TAKEAWAY Security rests on identity and explicit trust at every boundary — not network location.

SECURITY MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits behind most microservice security incidents.

MISCONCEPTION

REALITY

ANTI-PATTERNS TO AVOID

Inside the network is safe

Zero Trust verifies every request

A valid token allows everything

Check scopes and object ownership

JWTs are encrypted

Signed and encoded — anyone can read them

The gateway handles authorization

Each service must authorize too

Base64 Secrets are secure

Restrict access and use a vault

- ✗ Secrets in Git or images
- ✗ One admin role for everything
- ✗ Logging Authorization headers
- ✗ Implicit or password grant flows
- ✗ Skipping audience validation
- ✗ No audit of denied attempts

REMEMBER Authenticate early, authorize close to the action, audit decisions.

KEY TAKEAWAY Security is explicit trust at every hop — verified, least-privilege and auditable.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 Authentication vs. authorization?

2 What is OAuth2 primarily used for?

3 What does OpenID Connect add to OAuth2?

4 What is a JWT?

5 Which JWT claims should APIs validate?

6 What is Keycloak used for?

7 What is Microsoft Entra ID used for?

8 Why keep secrets out of source code?

9 What is object-level authorization?

10 Why secure internal APIs too?

MODULE 12 SUMMARY AND WHAT'S NEXT

From a larger attack surface to explicit, verified trust at every boundary.

YOU CAN NOW

- ✓ Apply Zero Trust and tell 401 from 403
- ✓ Explain OAuth2, OIDC, PKCE and client credentials
- ✓ Validate JWTs and enforce scopes and ownership in Spring Boot
- ✓ Deliver secrets at runtime and harden APIs
- ✓ Produce audit evidence and secure the pipeline

NEXT: CAPSTONE — PUTTING IT ALL TOGETHER

- 1 How do all twelve modules fit one platform?
- 2 What must the final demo prove?
- 3 Is the system ready for production?

BRING ALONG Your Lab 1–5 services — the capstone connects them end to end.

KEY TAKEAWAY The Capstone combines every module into one secure, observable, AI-assisted platform.